TINProofs C1-C5 Formal Verification Packet

Peer-review draft for the Lean 4 formalization of the five TIN/SNTC proof blocks.
Date: 2026-05-24
Build target: source ~/.elan/env && cd ~/Desktop/lean-proofs && lake build
Environment: Lean 4.29.1 with mathlib v4.29.1
Scope: 26 Lean modules, 1404 source lines, 124 top-level declarations, 76 theorem or lemma declarations.

Review Purpose

This packet is intended for a collaborator who needs to review the mathematical boundary of the formalization, not just see that the project compiles. Each problem section records the proof surface, the source modules, the main formal statements, and review notes. The appendix contains the complete Lean source for C1-C5.

Problem Map

- C1: Commodity Hull Theorem; files: TINProofs/C1/Defs.lean, TINProofs/C1/ActionAffine.lean, TINProofs/C1/SupportConcave.lean, TINProofs/C1/Crossover.lean
- C2: Stochastic Certification Asymmetry; files: TINProofs/C2/Defs.lean, TINProofs/C2/GapTiers.lean, TINProofs/C2/AsymmetryRatio.lean, TINProofs/C2/Absorbing.lean, TINProofs/C2/PoissonLimit.lean, TINProofs/C2/Expansion.lean
- C3: Lyapunov Radius of Validity; files: TINProofs/C3/Defs.lean, TINProofs/C3/EigenSandwich.lean, TINProofs/C3/VDotDecomp.lean, TINProofs/C3/QuadBound.lean, TINProofs/C3/RemainderBound.lean, TINProofs/C3/BallBound.lean, TINProofs/C3/RadiusOfValidity.lean
- C4: Temporal Transport Factorization; files: TINProofs/C4/Defs.lean, TINProofs/C4/Inclusion.lean, TINProofs/C4/Factorization.lean, TINProofs/C4/Monotonicity.lean, TINProofs/C4/BraessLocalization.lean
- C5: Three-Factor Sparse Law; files: TINProofs/C5/Defs.lean, TINProofs/C5/ThreeFactor.lean, TINProofs/C5/ChainProperties.lean, TINProofs/C5/Classification.lean

Current Verification Status

- The root import file TINProofs.lean imports C1 through C5.
- The full Lake build completed successfully after adding C1.
- The C1 source scan has no sorry, admit, or axiom declarations.
- C2-C5 retain their existing formal boundaries, including explicit hypotheses where the paper depends on external analytic or probabilistic facts.

C1. Commodity Hull Theorem

Mathematical Surface

- Finite path data are represented by PathData with exposure T, survival Q, and positivity bounds.
- A_pi(lambda) = -log Q_pi + lambda*T_pi and V_pi(lambda) = log Q_pi - lambda*T_pi.
- The support F(lambda) is Finset.inf' of path actions; the value V(lambda) is Finset.sup' of path values.
- The formal crossover slope is the sign required by the action equality proof.

Source Modules

- TINProofs/C1/Defs.lean: A path in the exposure-reliability plane.
- TINProofs/C1/ActionAffine.lean: Path action is affine in the commodity hazard rate.
- TINProofs/C1/SupportConcave.lean: The support function is concave: finite infimum of affine actions.
- TINProofs/C1/Crossover.lean: At the crossover slope, two paths have equal action.

Declarations for Review

TINProofs/C1/Defs.lean

structure PathData (TINProofs/C1/Defs.lean:10)

```
structure PathData where
T : ℝ
Q : ℝ
hT_nonneg : 0 ≤ T
hQ_pos : 0 < Q
hQ_le : Q ≤ 1
deriving DecidableEq
```

def PathData.action (TINProofs/C1/Defs.lean:19)

```
def PathData.action (p : PathData) (lam : ℝ) : ℝ :=
-Real.log p.Q + lam * p.T
```

def PathData.value (TINProofs/C1/Defs.lean:23)

```
def PathData.value (p : PathData) (lam : ℝ) : ℝ :=
Real.log p.Q - lam * p.T
```

def pathAction (TINProofs/C1/Defs.lean:27)

```
def pathAction (p : PathData) (lam : ℝ) : ℝ :=
p.action lam
```

def pathValue (TINProofs/C1/Defs.lean:31)

```
def pathValue (p : PathData) (lam : ℝ) : ℝ :=
p.value lam
```

structure CommodityHull (TINProofs/C1/Defs.lean:35)

```
structure CommodityHull where
paths : Finset PathData
hnonempty : paths.Nonempty
```

def CommodityHull.support (TINProofs/C1/Defs.lean:40)

```
def CommodityHull.support (ch : CommodityHull) (lam : ℝ) : ℝ :=
ch.paths.inf' ch.hnonempty (fun p => p.action lam)
```

def CommodityHull.valueFn (TINProofs/C1/Defs.lean:44)

```
def CommodityHull.valueFn (ch : CommodityHull) (lam : ℝ) : ℝ :=
ch.paths.sup' ch.hnonempty (fun p => p.value lam)
```

def crossoverSlope (TINProofs/C1/Defs.lean:48)

```
def crossoverSlope (pi pj : PathData) (_hT : pi.T ≠ pj.T) : ℝ :=
(Real.log pj.Q - Real.log pi.Q) / (pj.T - pi.T)
```

TINProofs/C1/ActionAffine.lean

theorem action_affine (TINProofs/C1/ActionAffine.lean:10)

```
theorem action_affine (p : PathData) (lam1 lam2 : ℝ) (a b : ℝ)
(_ha : 0 ≤ a) (_hb : 0 ≤ b) (hab : a + b = 1) :
p.action (a * lam1 + b * lam2) = a * p.action lam1 + b * p.action lam2
```

theorem action_convexOn (TINProofs/C1/ActionAffine.lean:22)

```
theorem action_convexOn (p : PathData) : ConvexOn ℝ univ p.action
```

theorem action_concaveOn (TINProofs/C1/ActionAffine.lean:29)

```
theorem action_concaveOn (p : PathData) : ConcaveOn ℝ univ p.action
```

theorem value_affine (TINProofs/C1/ActionAffine.lean:36)

```
theorem value_affine (p : PathData) (lam1 lam2 : ℝ) (a b : ℝ)
(_ha : 0 ≤ a) (_hb : 0 ≤ b) (hab : a + b = 1) :
p.value (a * lam1 + b * lam2) = a * p.value lam1 + b * p.value lam2
```

theorem value_convexOn (TINProofs/C1/ActionAffine.lean:48)

```
theorem value_convexOn (p : PathData) : ConvexOn ℝ univ p.value
```

theorem value_concaveOn (TINProofs/C1/ActionAffine.lean:55)

```
theorem value_concaveOn (p : PathData) : ConcaveOn ℝ univ p.value
```

theorem action_neg_value (TINProofs/C1/ActionAffine.lean:62)

```
theorem action_neg_value (p : PathData) (lam : ℝ) :
p.action lam = -(p.value lam)
```

TINProofs/C1/SupportConcave.lean

theorem support_concaveOn (TINProofs/C1/SupportConcave.lean:15)

```
theorem support_concaveOn (ch : CommodityHull) :
ConcaveOn ℝ univ ch.support
```

theorem valueFn_eq_neg_support (TINProofs/C1/SupportConcave.lean:37)

```
theorem valueFn_eq_neg_support (ch : CommodityHull) (lam : ℝ) :
ch.valueFn lam = -(ch.support lam)
```

theorem valueFn_convexOn (TINProofs/C1/SupportConcave.lean:60)

```
theorem valueFn_convexOn (ch : CommodityHull) :
ConvexOn ℝ univ ch.valueFn
```

TINProofs/C1/Crossover.lean

theorem action_eq_at_crossover (TINProofs/C1/Crossover.lean:10)

```
theorem action_eq_at_crossover (pi pj : PathData) (hT : pi.T ≠ pj.T) :
pi.action (crossoverSlope pi pj hT) = pj.action (crossoverSlope pi pj hT)
```

theorem action_lt_below_crossover (TINProofs/C1/Crossover.lean:18)

```
theorem action_lt_below_crossover (pi pj : PathData) (hT : pi.T < pj.T)
(lam : ℝ) (hlam : lam < crossoverSlope pi pj (ne_of_lt hT)) :
pj.action lam < pi.action lam
```

theorem action_gt_above_crossover (TINProofs/C1/Crossover.lean:28)

```
theorem action_gt_above_crossover (pi pj : PathData) (hT : pi.T < pj.T)
(lam : ℝ) (hlam : crossoverSlope pi pj (ne_of_lt hT) < lam) :
pi.action lam < pj.action lam
```

theorem legendre_duality (TINProofs/C1/Crossover.lean:38)

```
theorem legendre_duality (ch : CommodityHull) (lam : ℝ) :
ch.valueFn lam = -(ch.support lam) :=
valueFn_eq_neg_support ch lam

end TINProofs.C1

end
```

Review Notes

- Check the sign convention against the manuscript prose. The Lean proof uses the slope that makes action_eq_at_crossover true for A = -log Q + lambda*T.

C2. Stochastic Certification Asymmetry

Mathematical Surface

- Tier thresholds, delivery ratio, Binomial PMF/CDF/tails, asymmetry ratio, and achievable delivery ratios are formalized explicitly.
- The absorbing-tier and gap-tier claims are reduced to ceiling and finite-grid arithmetic.
- The Poisson-limit layer isolates distributional convergence and crossover existence conditions.
- The expansion layer records the external asymptotic input as a named hypothesis and proves the algebraic delivery-ratio consequence.

Source Modules

- TINProofs/C2/Defs.lean: A tier classification system with M tiers and M-1 thresholds.
- TINProofs/C2/GapTiers.lean: No integer X can satisfy xLower X < xUpper when xLower = xUpper.
- TINProofs/C2/AsymmetryRatio.lean: Tier cutoffs are positive because k > 0 and every threshold is positive.
- TINProofs/C2/Absorbing.lean: The last threshold, corresponding to the boundary above the lowest tier.
- TINProofs/C2/PoissonLimit.lean: Lower Poisson tail through m.
- TINProofs/C2/Expansion.lean: The asymptotic inverse of the reserve-cycle count.

Declarations for Review

TINProofs/C2/Defs.lean

structure TierSystem (TINProofs/C2/Defs.lean:14)

```
structure TierSystem where
M : ℕ
hM : 2 ≤ M
k : ℕ
hk : 0 < k
τ : Fin (M - 1) → ℝ
hτ_pos : ∀ i, 0 < τ i
hτ_lt_one : ∀ i, τ i < 1
hτ_strict_anti : StrictAnti τ
```

def deliveryRatio (TINProofs/C2/Defs.lean:25)

```
def deliveryRatio (k : ℕ) (X : ℕ) : ℝ :=
min ((X : ℝ) / (k : ℝ)) 1
```

def TierSystem.xMin (TINProofs/C2/Defs.lean:29)

```
def TierSystem.xMin (ts : TierSystem) (i : Fin (ts.M - 1)) : ℕ :=
⌈(ts.k : ℝ) * ts.τ i⌉₊
```

def binomialPMF (TINProofs/C2/Defs.lean:33)

```
def binomialPMF (n : ℕ) (p : ℝ) (j : ℕ) : ℝ :=
(n.choose j : ℝ) * p ^ j * (1 - p) ^ (n - j)
```

def binomialCDF (TINProofs/C2/Defs.lean:37)

```
def binomialCDF (n : ℕ) (p : ℝ) (m : ℕ) : ℝ :=
∑ j ∈ range (m + 1), binomialPMF n p j
```

def binomialUpperTail (TINProofs/C2/Defs.lean:41)

```
def binomialUpperTail (n : ℕ) (p : ℝ) (m : ℕ) : ℝ :=
∑ j ∈ Icc m n, binomialPMF n p j
```

def asymmetryRatio (TINProofs/C2/Defs.lean:46)

```
def asymmetryRatio (n : ℕ) (p : ℝ) (xLower xUpper : ℕ) : ℝ :=
binomialCDF n p (xLower - 1) / binomialUpperTail n p xUpper
```

def poissonPMF (TINProofs/C2/Defs.lean:50)

```
def poissonPMF (mu : ℝ) (j : ℕ) : ℝ :=
Real.exp (-mu) * mu ^ j / (j.factorial : ℝ)
```

def achievableDR (TINProofs/C2/Defs.lean:54)

```
def achievableDR (k : ℕ) : Finset ℝ :=
(range (k + 1)).image (fun (i : ℕ) => (i : ℝ) / (k : ℝ))
```

TINProofs/C2/GapTiers.lean

theorem no_achievable_in_gap (TINProofs/C2/GapTiers.lean:13)

```
theorem no_achievable_in_gap (xLower xUpper : ℕ) (h : xLower = xUpper) :
∀ X : ℕ, ¬(xLower ≤ X ∧ X < xUpper)
```

theorem gap_tier (TINProofs/C2/GapTiers.lean:20)

```
theorem gap_tier (ts : TierSystem)
(i j : Fin (ts.M - 1))
(h_gap : ts.xMin i = ts.xMin j) :
∀ X : ℕ, ¬(ts.xMin i ≤ X ∧ X < ts.xMin j) :=
no_achievable_in_gap _ _ h_gap
```

TINProofs/C2/AsymmetryRatio.lean

theorem TierSystem.xMin_pos (TINProofs/C2/AsymmetryRatio.lean:14)

```
theorem TierSystem.xMin_pos (ts : TierSystem) (i : Fin (ts.M - 1)) :
0 < ts.xMin i
```

theorem binomialCDF_pred_eq_sum_range (TINProofs/C2/AsymmetryRatio.lean:22)

```
theorem binomialCDF_pred_eq_sum_range (n : ℕ) (p : ℝ) (x : ℕ) (hx : 0 < x) :
binomialCDF n p (x - 1) = ∑ j ∈ range x, binomialPMF n p j
```

def tierAsymmetryRatio (TINProofs/C2/AsymmetryRatio.lean:28)

```
def tierAsymmetryRatio (ts : TierSystem) (n : ℕ) (p : ℝ)
(T Tpred : Fin (ts.M - 1)) : ℝ :=
asymmetryRatio n p (ts.xMin T) (ts.xMin Tpred)
```

theorem tierAsymmetryRatio_eq_binomial_sums (TINProofs/C2/AsymmetryRatio.lean:34)

```
theorem tierAsymmetryRatio_eq_binomial_sums (ts : TierSystem) (n : ℕ) (p : ℝ)
(T Tpred : Fin (ts.M - 1)) :
tierAsymmetryRatio ts n p T Tpred =
(∑ j ∈ range (ts.xMin T), binomialPMF n p j) /
(∑ j ∈ Icc (ts.xMin Tpred) n, binomialPMF n p j)
```

TINProofs/C2/Absorbing.lean

def TierSystem.lowestThreshold (TINProofs/C2/Absorbing.lean:16)

```
def TierSystem.lowestThreshold (ts : TierSystem) : Fin (ts.M - 1) :=
⟨ts.M - 2, by
have hM : 2 ≤ ts.M := ts.hM
omega⟩
```

theorem ceil_mean_lt_xMin_of_lt_sub_one (TINProofs/C2/Absorbing.lean:24)

```
theorem ceil_mean_lt_xMin_of_lt_sub_one (ts : TierSystem) (i : Fin (ts.M - 1))
(n : ℕ) (p : ℝ) (hp : 0 ≤ p)
(h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
⌈(n : ℝ) * p⌉₊ < ts.xMin i
```

theorem ceil_mean_le_xMin_of_lt_sub_one (TINProofs/C2/Absorbing.lean:37)

```
theorem ceil_mean_le_xMin_of_lt_sub_one (ts : TierSystem) (i : Fin (ts.M - 1))
(n : ℕ) (p : ℝ) (hp : 0 ≤ p)
(h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
⌈(n : ℝ) * p⌉₊ ≤ ts.xMin i :=
le_of_lt (ceil_mean_lt_xMin_of_lt_sub_one ts i n p hp h)

/-- Conditional median consequence: once the standard Binomial median upper
bound is supplied, the median is below the tier cutoff. -/
```

theorem median_lt_xMin_of_median_le_ceil_mean (TINProofs/C2/Absorbing.lean:45)

```
theorem median_lt_xMin_of_median_le_ceil_mean (ts : TierSystem)
(i : Fin (ts.M - 1)) (n median : ℕ) (p : ℝ) (hp : 0 ≤ p)
(hmedian : median ≤ ⌈(n : ℝ) * p⌉₊)
(h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
median < ts.xMin i :=
lt_of_le_of_lt hmedian (ceil_mean_lt_xMin_of_lt_sub_one ts i n p hp h)

/-- Conditional mode consequence: if the mode is no larger than such a median,
then the mode is also below the tier cutoff. -/
```

theorem mode_lt_xMin_of_mode_le_median (TINProofs/C2/Absorbing.lean:54)

```
theorem mode_lt_xMin_of_mode_le_median (ts : TierSystem)
(i : Fin (ts.M - 1)) (n mode median : ℕ) (p : ℝ) (hp : 0 ≤ p)
(hmode : mode ≤ median)
(hmedian : median ≤ ⌈(n : ℝ) * p⌉₊)
(h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
mode < ts.xMin i :=
lt_of_le_of_lt hmode
(median_lt_xMin_of_median_le_ceil_mean ts i n median p hp hmedian h)

/-- Lowest-tier specialization of the ceiling cutoff bound. -/
```

theorem absorbing_lowest_tier_ceiling_lt_cutoff (TINProofs/C2/Absorbing.lean:64)

```
theorem absorbing_lowest_tier_ceiling_lt_cutoff (ts : TierSystem)
(n : ℕ) (p : ℝ) (hp : 0 ≤ p)
(h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ ts.lowestThreshold - 1) :
⌈(n : ℝ) * p⌉₊ < ts.xMin ts.lowestThreshold :=
ceil_mean_lt_xMin_of_lt_sub_one ts ts.lowestThreshold n p hp h

/-- Lowest-tier specialization with the Binomial median upper bound supplied
as an explicit hypothesis. -/
```

theorem absorbing_lowest_tier_median_lt_cutoff (TINProofs/C2/Absorbing.lean:72)

```
theorem absorbing_lowest_tier_median_lt_cutoff (ts : TierSystem)
(n median : ℕ) (p : ℝ) (hp : 0 ≤ p)
(hmedian : median ≤ ⌈(n : ℝ) * p⌉₊)
(h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ ts.lowestThreshold - 1) :
median < ts.xMin ts.lowestThreshold :=
median_lt_xMin_of_median_le_ceil_mean
ts ts.lowestThreshold n median p hp hmedian h
```

end

TINProofs/C2/PoissonLimit.lean

def poissonLowerTail (TINProofs/C2/PoissonLimit.lean:17)

```
def poissonLowerTail (mu : ℝ) (m : ℕ) : ℝ :=
∑ j ∈ range (m + 1), poissonPMF mu j
```

def poissonUpperTail (TINProofs/C2/PoissonLimit.lean:21)

```
def poissonUpperTail (mu : ℝ) (k : ℕ) : ℝ :=
∑' j : ℕ, if k ≤ j then poissonPMF mu j else 0
```

def poissonCrossoverCondition (TINProofs/C2/PoissonLimit.lean:26)

```
def poissonCrossoverCondition (k : ℕ) (mu : ℝ) : Prop :=
poissonLowerTail mu (k - 2) = poissonUpperTail mu k
```

theorem poissonCrossoverCondition_iff (TINProofs/C2/PoissonLimit.lean:30)

```
theorem poissonCrossoverCondition_iff (k : ℕ) (mu : ℝ) :
poissonCrossoverCondition k mu ↔
(∑ j ∈ range (k - 2 + 1), poissonPMF mu j) =
(∑' j : ℕ, if k ≤ j then poissonPMF mu j else 0)
```

theorem binomialPMF_tendsto_poissonPMF (TINProofs/C2/PoissonLimit.lean:39)

```
theorem binomialPMF_tendsto_poissonPMF (k : ℕ) {r : ℝ} {p : ℕ → ℝ}
(hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
Tendsto (fun n => binomialPMF n (p n) k) atTop (𝓝 (poissonPMF r k))
```

theorem binomialLowerTail_tendsto_poissonLowerTail (TINProofs/C2/PoissonLimit.lean:47)

```
theorem binomialLowerTail_tendsto_poissonLowerTail (m : ℕ) {r : ℝ} {p : ℕ → ℝ}
(hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
Tendsto (fun n => ∑ j ∈ range (m + 1), binomialPMF n (p n) j)
atTop (𝓝 (poissonLowerTail r m))
```

theorem binomialPMF_sum_range_eq_one (TINProofs/C2/PoissonLimit.lean:56)

```
theorem binomialPMF_sum_range_eq_one (n : ℕ) (p : ℝ) :
∑ j ∈ range (n + 1), binomialPMF n p j = 1
```

lemma Icc_eq_Ico_succ_nat (TINProofs/C2/PoissonLimit.lean:65)

```
lemma Icc_eq_Ico_succ_nat (k n : ℕ) : Icc k n = Ico k (n + 1)
```

theorem binomialUpperTail_eq_one_sub_lower (TINProofs/C2/PoissonLimit.lean:71)

```
theorem binomialUpperTail_eq_one_sub_lower (n : ℕ) (p : ℝ) (k : ℕ)
(hk : k ≤ n + 1) :
binomialUpperTail n p k = 1 - ∑ j ∈ range k, binomialPMF n p j
```

theorem hasSum_poissonPMF (TINProofs/C2/PoissonLimit.lean:81)

```
theorem hasSum_poissonPMF (mu : ℝ) : HasSum (fun j : ℕ => poissonPMF mu j) 1
```

theorem tsum_ite_ge_eq_nat_add (TINProofs/C2/PoissonLimit.lean:93)

```
theorem tsum_ite_ge_eq_nat_add (f : ℕ → ℝ) (k : ℕ) :
(∑' j : ℕ, if k ≤ j then f j else 0) = ∑' i : ℕ, f (i + k)
```

theorem poissonUpperTail_eq_one_sub_lower (TINProofs/C2/PoissonLimit.lean:110)

```
theorem poissonUpperTail_eq_one_sub_lower (mu : ℝ) (k : ℕ) :
poissonUpperTail mu k = 1 - ∑ j ∈ range k, poissonPMF mu j
```

theorem poissonPMF_zero_zero (TINProofs/C2/PoissonLimit.lean:120)

```
theorem poissonPMF_zero_zero : poissonPMF 0 0 = 1
```

theorem poissonPMF_zero_of_pos (TINProofs/C2/PoissonLimit.lean:124)

```
theorem poissonPMF_zero_of_pos (j : ℕ) (hj : 0 < j) : poissonPMF 0 j = 0
```

theorem sum_range_poissonPMF_zero_eq_one (TINProofs/C2/PoissonLimit.lean:128)

```
theorem sum_range_poissonPMF_zero_eq_one (k : ℕ) (hk : 0 < k) :
∑ j ∈ range k, poissonPMF 0 j = 1
```

theorem poissonUpperTail_zero_of_pos (TINProofs/C2/PoissonLimit.lean:139)

```
theorem poissonUpperTail_zero_of_pos (k : ℕ) (hk : 0 < k) :
poissonUpperTail 0 k = 0
```

theorem poissonLowerTail_zero_eq_one (TINProofs/C2/PoissonLimit.lean:145)

```
theorem poissonLowerTail_zero_eq_one (m : ℕ) :
poissonLowerTail 0 m = 1
```

theorem binomialFiniteLowerTail_tendsto_poissonFiniteLowerTail (TINProofs/C2/PoissonLimit.lean:153)

```
theorem binomialFiniteLowerTail_tendsto_poissonFiniteLowerTail
(k : ℕ) {r : ℝ} {p : ℕ → ℝ}
(hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
Tendsto (fun n => ∑ j ∈ range k, binomialPMF n (p n) j)
atTop (𝓝 (∑ j ∈ range k, poissonPMF r j))
```

theorem binomialUpperTail_tendsto_poissonUpperTail (TINProofs/C2/PoissonLimit.lean:162)

```
theorem binomialUpperTail_tendsto_poissonUpperTail
(k : ℕ) {r : ℝ} {p : ℕ → ℝ}
(hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
Tendsto (fun n => binomialUpperTail n (p n) k) atTop (𝓝 (poissonUpperTail r k))
```

theorem continuous_poissonPMF (TINProofs/C2/PoissonLimit.lean:179)

```
theorem continuous_poissonPMF (j : ℕ) : Continuous fun mu : ℝ => poissonPMF mu j
```

theorem continuous_poissonLowerTail (TINProofs/C2/PoissonLimit.lean:184)

theorem continuous_poissonLowerTail (m : ℕ) : Continuous fun mu : ℝ => poissonLowerTail mu m

theorem continuous_poissonFiniteLowerTail (TINProofs/C2/PoissonLimit.lean:189)

theorem continuous_poissonFiniteLowerTail (k : ℕ) :
Continuous fun mu : ℝ => ∑ j ∈ range k, poissonPMF mu j

theorem continuous_poissonUpperTail (TINProofs/C2/PoissonLimit.lean:194)

theorem continuous_poissonUpperTail (k : ℕ) :
Continuous fun mu : ℝ => poissonUpperTail mu k

theorem poissonCrossover_exists_of_bracket (TINProofs/C2/PoissonLimit.lean:205)

theorem poissonCrossover_exists_of_bracket (k : ℕ) (a b : ℝ) (hab : a ≤ b)
(ha : poissonUpperTail a k ≤ poissonLowerTail a (k - 2))
(hb : poissonLowerTail b (k - 2) ≤ poissonUpperTail b k) :
∃ mu ∈ Set.Icc a b, poissonCrossoverCondition k mu

theorem poissonCrossover_exists_of_reverse_at (TINProofs/C2/PoissonLimit.lean:223)

theorem poissonCrossover_exists_of_reverse_at (k : ℕ) (b : ℝ)
(hk : 2 ≤ k) (hb0 : 0 ≤ b)
(hb : poissonLowerTail b (k - 2) ≤ poissonUpperTail b k) :
∃ mu ∈ Set.Icc 0 b, poissonCrossoverCondition k mu

TINProofs/C2/Expansion.lean

def invK (TINProofs/C2/Expansion.lean:15)

def invK (k : ℕ) : ℝ :=
((k : ℝ)⁻¹)

def invKSq (TINProofs/C2/Expansion.lean:19)

def invKSq (k : ℕ) : ℝ :=
invK k ^ 2

def lambdaCenter (TINProofs/C2/Expansion.lean:24)

def lambdaCenter (k : ℕ) : ℝ :=
(k : ℝ) - (5 / 6 : ℝ)

def drStar (TINProofs/C2/Expansion.lean:28)

def drStar (lambdaStar : ℕ → ℝ) (k : ℕ) : ℝ :=
lambdaStar k / (k : ℝ)

def drTarget (TINProofs/C2/Expansion.lean:32)

def drTarget (k : ℕ) : ℝ :=
1 - (5 / 6 : ℝ) / (k : ℝ)

def selectsPoissonCrossover (TINProofs/C2/Expansion.lean:36)

def selectsPoissonCrossover (lambdaStar : ℕ → ℝ) : Prop :=
∀ᶠ k in atTop, poissonCrossoverCondition k (lambdaStar k)

def adellJodraAsymptoticInput (TINProofs/C2/Expansion.lean:41)

def adellJodraAsymptoticInput (lambdaStar : ℕ → ℝ) : Prop :=
(fun k : ℕ => lambdaStar k - lambdaCenter k) =O[atTop] invK

theorem drStar_expansion_of_lambda_expansion (TINProofs/C2/Expansion.lean:47)

theorem drStar_expansion_of_lambda_expansion (lambdaStar : ℕ → ℝ)
(_hCross : selectsPoissonCrossover lambdaStar)
(hLam : adellJodraAsymptoticInput lambdaStar) :
(fun k : ℕ => drStar lambdaStar k - drTarget k) =O[atTop] invKSq

Review Notes

- Median and mode facts remain explicit hypotheses rather than first-class Binomial median/mode definitions.
- The asymptotic theorem depends on the named Adell-Jodra-style input; review whether this is the intended trust boundary.

C3. Lyapunov Radius of Validity

Mathematical Surface

- The setup stores eigenvalue, decay, coupling, and cubic-remainder constants as real inequalities.
- The proof stack separates eigenvalue sandwiching, Vdot decomposition, quadratic lower bounds, remainder bounds, and ball-local derivative control.
- The main radius theorem proves nonpositive Vdot inside the critical radius and then derives sublevel-set corollaries.

Source Modules

- TINProofs/C3/Defs.lean: Scalar data used by the Lyapunov radius estimate.
- TINProofs/C3/EigenSandwich.lean: Lower Rayleigh-quotient bound for the Lyapunov quadratic form.
- TINProofs/C3/VDotDecomp.lean: Pointwise derivative decomposition: `Vdot = -delta^T Q delta + 2 delta^T A R(delta)`.
- TINProofs/C3/QuadBound.lean: The Lyapunov matrix quadratic form is bounded below by `q * rho^2`.
- TINProofs/C3/RemainderBound.lean: Absolute bound for the nonlinear Lyapunov remainder term.
- TINProofs/C3/BallBound.lean: Combining the decomposition, quadratic lower bound, and remainder bound.
- TINProofs/C3/RadiusOfValidity.lean: The critical radius is positive under the setup hypotheses.

Declarations for Review

TINProofs/C3/Defs.lean

structure LyapunovSetup (TINProofs/C3/Defs.lean:14)

```
structure LyapunovSetup where
alphaMin : ℝ
alphaMax : ℝ
q : ℝ
r0 : ℝ
opNormA : ℝ
M : ℝ
hAlphaMin_pos : 0 < alphaMin
hAlphaMax_pos : 0 < alphaMax
hq_pos : 0 < q
hr0_pos : 0 < r0
hOpNormA_pos : 0 < opNormA
hM_pos : 0 < M
```

def criticalRadius (TINProofs/C3/Defs.lean:29)

```
def criticalRadius (S : LyapunovSetup) : ℝ :=
min S.r0 (S.q / (2 * S.opNormA * S.M))
```

def lyapunovDecayRate (TINProofs/C3/Defs.lean:33)

```
def lyapunovDecayRate (S : LyapunovSetup) : ℝ :=
S.q / (2 * S.alphaMax)
```

structure LyapunovPoint (TINProofs/C3/Defs.lean:42)

```
structure LyapunovPoint (S : LyapunovSetup) where
rho : ℝ
V : ℝ
Vdot : ℝ
quad : ℝ
remainder : ℝ
hrho_nonneg : 0 ≤ rho
eigenSandwich : S.alphaMin * rho ^ 2 ≤ V ∧ V ≤ S.alphaMax * rho ^ 2
vdotDecomp : Vdot = -quad + remainder
quadLower : S.q * rho ^ 2 ≤ quad
remainderAbsBound : |remainder| ≤ S.opNormA * S.M * rho ^ 3
```

TINProofs/C3/EigenSandwich.lean

theorem eigen_sandwich_lower (TINProofs/C3/EigenSandwich.lean:13)

```
theorem eigen_sandwich_lower :
S.alphaMin * P.rho ^ 2 ≤ P.V :=
P.eigenSandwich.1

/-- Upper Rayleigh-quotient bound for the Lyapunov quadratic form. -/
```

theorem eigen_sandwich_upper (TINProofs/C3/EigenSandwich.lean:18)

```
theorem eigen_sandwich_upper :
P.V ≤ S.alphaMax * P.rho ^ 2 :=
P.eigenSandwich.2

/-- The full Lyapunov eigenvalue sandwich. -/
```

theorem eigen_sandwich (TINProofs/C3/EigenSandwich.lean:23)

```
theorem eigen_sandwich :
S.alphaMin * P.rho ^ 2 ≤ P.V ∧ P.V ≤ S.alphaMax * P.rho ^ 2 :=
P.eigenSandwich

end TINProofs.C3
```

```
end
```

TINProofs/C3/VDotDecomp.lean

theorem vdot_decomposition (TINProofs/C3/VDotDecomp.lean:16)

```
theorem vdot_decomposition :
P.Vdot = -P.quad + P.remainder :=
P.vdotDecomp

end TINProofs.C3
```

```
end
```

TINProofs/C3/QuadBound.lean

theorem quadratic_term_lower (TINProofs/C3/QuadBound.lean:13)

```
theorem quadratic_term_lower :
S.q * P.rho ^ 2 ≤ P.quad :=
P.quadLower

/-- The negative quadratic term is bounded above by `-q * rho^2`. -/
```

theorem negative_quadratic_term_upper (TINProofs/C3/QuadBound.lean:18)

```
theorem negative_quadratic_term_upper :
-P.quad ≤ -(S.q * P.rho ^ 2) :=
neg_le_neg P.quadLower

end TINProofs.C3
```

```
end
```

TINProofs/C3/RemainderBound.lean

theorem remainder_abs_bound (TINProofs/C3/RemainderBound.lean:13)

```
theorem remainder_abs_bound :
|P.remainder| ≤ S.opNormA * S.M * P.rho ^ 3 :=
P.remainderAbsBound

/-- One-sided form of the nonlinear remainder bound. -/
```

theorem remainder_upper_bound (TINProofs/C3/RemainderBound.lean:18)

```
theorem remainder_upper_bound :
P.remainder ≤ S.opNormA * S.M * P.rho ^ 3 :=
le_trans (le_abs_self P.remainder) P.remainderAbsBound

end TINProofs.C3
```

```
end
```

TINProofs/C3/BallBound.lean

theorem vdot_le_quadratic_plus_cubic (TINProofs/C3/BallBound.lean:15)

```
theorem vdot_le_quadratic_plus_cubic :
P.Vdot ≤ -S.q * P.rho ^ 2 + S.opNormA * S.M * P.rho ^ 3
```

theorem vdot_bound_on_ball (TINProofs/C3/BallBound.lean:25)

```
theorem vdot_bound_on_ball (hball : P.rho < criticalRadius S) :
P.Vdot ≤ -(S.q / 2) * P.rho ^ 2
```

TINProofs/C3/RadiusOfValidity.lean

theorem criticalRadius_pos (TINProofs/C3/RadiusOfValidity.lean:14)

```
theorem criticalRadius_pos (S : LyapunovSetup) :
0 < criticalRadius S
```

theorem lyapunov_radius_of_validity (TINProofs/C3/RadiusOfValidity.lean:26)

```
theorem lyapunov_radius_of_validity (hball : P.rho < criticalRadius S) :
P.Vdot ≤ -(lyapunovDecayRate S) * P.V
```

theorem sublevel_inside_critical_ball (TINProofs/C3/RadiusOfValidity.lean:57)

```
theorem sublevel_inside_critical_ball {V0 : ℝ}
(hV : P.V ≤ V0)
(hV0 : V0 < S.alphaMin * criticalRadius S ^ 2) :
P.rho < criticalRadius S
```

theorem sublevel_boundary_vdot_nonpos (TINProofs/C3/RadiusOfValidity.lean:74)

```
theorem sublevel_boundary_vdot_nonpos {V0 : ℝ}
(hV : P.V = V0)
(hV0 : V0 < S.alphaMin * criticalRadius S ^ 2) :
P.Vdot ≤ 0
```

Review Notes

- All analytic inputs are scalar inequalities in the setup; review whether the abstraction matches the paper's intended matrix and Taylor hypotheses.

C4. Temporal Transport Factorization

Mathematical Surface

- Temporal transport is represented over abstract measurable events D, F, and E with delivery included in feasibility and efficiency.
- The factorization proves DR = ST * eta under nonzero feasibility mass, both in ENNReal and real-valued forms.
- Augmentation monotonicity is isolated to the feasible-reachability factor, and Braess-style decreases are localized in efficiency.

Source Modules

- TINProofs/C4/Defs.lean: A temporal transport instance: a probability space with delivery and feasibility events.
- TINProofs/C4/Inclusion.lean: A delivered but infeasible sample cannot occur.
- TINProofs/C4/Factorization.lean: Exact `ENNReal` factorization: `DR = S_T * eta`.
- TINProofs/C4/Monotonicity.lean: `S_T` is monotone under augmentation.
- TINProofs/C4/BraessLocalization.lean: If delivery ratio decreases under augmentation, efficiency decreases.

Declarations for Review

TINProofs/C4/Defs.lean

structure TemporalTransport (TINProofs/C4/Defs.lean:19)

```
structure TemporalTransport (Ω : Type*) [MeasurableSpace Ω] where
μ : Measure Ω
hprob : IsProbabilityMeasure μ
D : Set Ω
F : Set Ω
hD : MeasurableSet D
hF : MeasurableSet F
h_incl : D ⊆ F
```

def DR (TINProofs/C4/Defs.lean:35)

```
def DR {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ENNReal :=
tt.μ tt.D
```

def ST (TINProofs/C4/Defs.lean:39)

```
def ST {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ENNReal :=
tt.μ tt.F
```

def eta (TINProofs/C4/Defs.lean:43)

```
def eta {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ENNReal :=
tt.μ tt.D / tt.μ tt.F
```

def DRReal (TINProofs/C4/Defs.lean:47)

```
def DRReal {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ℝ :=
(tt.DR).toReal
```

def STReal (TINProofs/C4/Defs.lean:51)

```
def STReal {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ℝ :=
(tt.ST).toReal
```

def etaReal (TINProofs/C4/Defs.lean:55)

```
def etaReal {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ℝ :=
tt.DRReal / tt.STReal
```

structure Augmentation (TINProofs/C4/Defs.lean:64)

```
structure Augmentation {Ω : Type*} [MeasurableSpace Ω]
(tt tt' : TemporalTransport Ω) where
h_F_mono : tt.F ⊆ tt'.F
```

TINProofs/C4/Inclusion.lean

theorem delivery_inter_infeasible_eq_empty (TINProofs/C4/Inclusion.lean:15)

```
theorem delivery_inter_infeasible_eq_empty (tt : TemporalTransport Ω) :
tt.D ∩ tt.Fᶜ = ∅
```

theorem delivery_zero_when_infeasible (TINProofs/C4/Inclusion.lean:25)

```
theorem delivery_zero_when_infeasible (tt : TemporalTransport Ω) :
tt.μ (tt.D ∩ tt.Fᶜ) = 0
```

theorem delivery_measure_le_feasible (TINProofs/C4/Inclusion.lean:31)

```
theorem delivery_measure_le_feasible (tt : TemporalTransport Ω) :
tt.μ tt.D ≤ tt.μ tt.F :=
measure_mono tt.h_incl

end TINProofs.C4
```

```
end
```

TINProofs/C4/Factorization.lean

theorem exact_factorization (TINProofs/C4/Factorization.lean:16)

```
theorem exact_factorization (tt : TemporalTransport Ω) (hF : tt.ST ≠ 0) :
tt.DR = tt.ST * tt.eta
```

theorem exact_factorization_real (TINProofs/C4/Factorization.lean:24)

```
theorem exact_factorization_real (tt : TemporalTransport Ω) (hF : tt.STReal ≠ 0) :
tt.DRReal = tt.STReal * tt.etaReal
```

TINProofs/C4/Monotonicity.lean

theorem st_monotone (TINProofs/C4/Monotonicity.lean:15)

```
theorem st_monotone (tt tt' : TemporalTransport Ω)
(h_same_measure : tt.μ = tt'.μ)
(aug : Augmentation tt tt') :
tt.ST ≤ tt'.ST
```

theorem stReal_monotone (TINProofs/C4/Monotonicity.lean:24)

```
theorem stReal_monotone (tt tt' : TemporalTransport Ω)
(h_same_measure : tt.μ = tt'.μ)
(aug : Augmentation tt tt') :
tt.STReal ≤ tt'.STReal
```

TINProofs/C4/BraessLocalization.lean

theorem braess_in_eta (TINProofs/C4/BraessLocalization.lean:16)

```
theorem braess_in_eta (tt tt' : TemporalTransport Ω)
(h_same_measure : tt.μ = tt'.μ)
(aug : Augmentation tt tt')
(hF : 0 < (tt.μ tt.F).toReal)
(_hF' : 0 < (tt'.μ tt'.F).toReal)
(h_dr_decrease : (tt'.μ tt'.D).toReal < (tt.μ tt.D).toReal) :
(tt'.μ tt'.D).toReal / (tt'.μ tt'.F).toReal <
(tt.μ tt.D).toReal / (tt.μ tt.F).toReal
```

theorem braess_in_etaReal (TINProofs/C4/BraessLocalization.lean:31)

```
theorem braess_in_etaReal (tt tt' : TemporalTransport Ω)
(h_same_measure : tt.μ = tt'.μ)
(aug : Augmentation tt tt')
(hF : 0 < tt.STReal)
(_hF' : 0 < tt'.STReal)
(h_dr_decrease : tt'.DRReal < tt.DRReal) :
tt'.etaReal < tt.etaReal
```

Review Notes

- The model is event-level rather than graph-level. Peer review should check whether the event abstraction captures every manuscript dependency.

C5. Three-Factor Sparse Law

Mathematical Surface

- The scalar sparse-law setup stores S_T, eta, E_H, Lyapunov exponent, DR, and their bounds.
- etaLyap = exp(E_H * lyap), Phi = eta / etaLyap, and the main theorem proves DR = S_T * etaLyap * Phi.
- Additional lemmas capture positivity, etaLyap < 1, and morphology vocabulary for trap/cluster classification.

Source Modules

- TINProofs/C5/Defs.lean: Scalar parameters for the three-factor sparse law.
- TINProofs/C5/ThreeFactor.lean: The three-factor sparse law: `DR = S_T * exp(E[H] * lambda) * Phi`.
- TINProofs/C5/ChainProperties.lean: Chain attenuation is positive.
- TINProofs/C5/Classification.lean: Finite-difference morphology slope for the routing residual.

Declarations for Review

TINProofs/C5/Defs.lean

structure SparseLawSetup (TINProofs/C5/Defs.lean:13)

```
structure SparseLawSetup where
S_T : ℝ
eta : ℝ
E_H : ℝ
lyap : ℝ
hST_pos : 0 < S_T
hST_le : S_T ≤ 1
heta_nonneg : 0 ≤ eta
heta_le : eta ≤ 1
hEH_pos : 0 < E_H
hlyap_neg : lyap < 0
DR : ℝ
hDR : DR = S_T * eta
```

def SparseLawSetup.etaLyap (TINProofs/C5/Defs.lean:28)

```
def SparseLawSetup.etaLyap (s : SparseLawSetup) : ℝ :=
Real.exp (s.E_H * s.lyap)
```

def SparseLawSetup.Phi (TINProofs/C5/Defs.lean:32)

```
def SparseLawSetup.Phi (s : SparseLawSetup) : ℝ :=
s.eta / s.etaLyap
```

TINProofs/C5/ThreeFactor.lean

theorem three_factor (TINProofs/C5/ThreeFactor.lean:13)

```
theorem three_factor (s : SparseLawSetup) :
s.DR = s.S_T * s.etaLyap * s.Phi
```

TINProofs/C5/ChainProperties.lean

theorem etaLyap_pos (TINProofs/C5/ChainProperties.lean:13)

```
theorem etaLyap_pos (s : SparseLawSetup) : 0 < s.etaLyap
```

theorem etaLyap_lt_one (TINProofs/C5/ChainProperties.lean:18)

```
theorem etaLyap_lt_one (s : SparseLawSetup) : s.etaLyap < 1
```

theorem phi_pos (TINProofs/C5/ChainProperties.lean:24)

```
theorem phi_pos (s : SparseLawSetup) (heta : 0 < s.eta) : 0 < s.Phi
```

theorem lyapunov_exponent_neg (TINProofs/C5/ChainProperties.lean:29)

```
theorem lyapunov_exponent_neg {n : ℕ} (ps : Fin n → ℝ)
(hps_pos : ∀ i, 0 < ps i) (hps_lt : ∀ i, ps i < 1)
(hn : 0 < n) :
(∑ i, Real.log (ps i)) / (n : ℝ) < 0
```

TINProofs/C5/Classification.lean

def morphologySlope (TINProofs/C5/Classification.lean:13)

```
def morphologySlope (phi1 phi2 : ℝ) (eh1 eh2 : ℝ)
(_hphi1 : 0 < phi1) (_hphi2 : 0 < phi2) (_heh : eh1 ≠ eh2) : ℝ :=
(Real.log phi2 - Real.log phi1) / (eh2 - eh1)
```

def isTrap (TINProofs/C5/Classification.lean:18)

```
def isTrap (gamma : ℝ) : Prop :=
gamma < 0
```

def isCluster (TINProofs/C5/Classification.lean:22)

```
def isCluster (gamma : ℝ) : Prop :=
0 < gamma
```

Review Notes

- The formal theorem is scalar and algebraic. Review whether any probabilistic interpretation should remain outside this theorem statement.

Appendix: Complete Lean Source

The listings below are the full C1-C5 Lean modules included in the root TINProofs import surface. Line numbers are local to each file.

Appendix C1: Commodity Hull Theorem

TINProofs/C1/Defs.lean

```
import Mathlib

open Real Set

noncomputable section

namespace TINProofs.C1

/-- A path in the exposure-reliability plane. -/
structure PathData where
  T : ℝ
  Q : ℝ
  hT_nonneg : 0 ≤ T
  hQ_pos : 0 < Q
  hQ_le : Q ≤ 1
  deriving DecidableEq

/-- Path action at commodity hazard rate `lam`. -/
def PathData.action (p : PathData) (lam : ℝ) : ℝ :=
  -Real.log p.Q + lam * p.T

/-- Path value at commodity hazard rate `lam`. -/
def PathData.value (p : PathData) (lam : ℝ) : ℝ :=
  Real.log p.Q - lam * p.T

/-- Alias for path action at commodity hazard rate `lam`. -/
def pathAction (p : PathData) (lam : ℝ) : ℝ :=
  p.action lam

/-- Alias for path value at commodity hazard rate `lam`. -/
def pathValue (p : PathData) (lam : ℝ) : ℝ :=
  p.value lam

/-- A nonempty finite set of feasible paths. -/
structure CommodityHull where
  paths : Finset PathData
  hnonempty : paths.Nonempty

/-- Support function: the minimum path action at `lam`. -/
def CommodityHull.support (ch : CommodityHull) (lam : ℝ) : ℝ :=
  ch.paths.inf' ch.hnonempty (fun p => p.action lam)

/-- Value function: the maximum path value at `lam`. -/
def CommodityHull.valueFn (ch : CommodityHull) (lam : ℝ) : ℝ :=
  ch.paths.sup' ch.hnonempty (fun p => p.value lam)

/-- Crossover slope between two paths with distinct exposure times. -/
def crossoverSlope (pi pj : PathData) (_hT : pi.T ≠ pj.T) : ℝ :=
  (Real.log pj.Q - Real.log pi.Q) / (pj.T - pi.T)

end TINProofs.C1

end
```

TINProofs/C1/ActionAffine.lean

```
import TINProofs.C1.Defs

open Real Set

noncomputable section

namespace TINProofs.C1

/-- Path action is affine in the commodity hazard rate. -/
theorem action_affine (p : PathData) (lam1 lam2 : ℝ) (a b : ℝ)
    (_ha : 0 ≤ a) (_hb : 0 ≤ b) (hab : a + b = 1) :
    p.action (a * lam1 + b * lam2) = a * p.action lam1 + b * p.action lam2 := by
  unfold PathData.action
  have hconst : -Real.log p.Q = a * (-Real.log p.Q) + b * (-Real.log p.Q) := by
    calc
      -Real.log p.Q = (a + b) * (-Real.log p.Q) := by rw [hab]; ring
      _ = a * (-Real.log p.Q) + b * (-Real.log p.Q) := by ring
  conv_lhs => rw [hconst]
  ring

/-- Path action is convex on all hazard rates. -/
theorem action_convexOn (p : PathData) : ConvexOn ℝ univ p.action := by
  constructor
  · exact convex_univ
  · intro x _hx y _hy a b ha hb hab
    simpa [smul_eq_mul] using le_of_eq (action_affine p x y a b ha hb hab)
```

```
27
28  /-- Path action is concave on all hazard rates. -/
29  theorem action_concaveOn (p : PathData) : ConcaveOn ℝ univ p.action := by
30    constructor
31    · exact convex_univ
32    · intro x _hx y _hy a b ha hb hab
33      simpa [smul_eq_mul] using le_of_eq (action_affine p x y a b ha hb hab).symm
34
35  /-- Per-path value is affine in the commodity hazard rate. -/
36  theorem value_affine (p : PathData) (lam1 lam2 : ℝ) (a b : ℝ)
37      (_ha : 0 ≤ a) (_hb : 0 ≤ b) (hab : a + b = 1) :
38      p.value (a * lam1 + b * lam2) = a * p.value lam1 + b * p.value lam2 := by
39    unfold PathData.value
40    have hconst : Real.log p.Q = a * Real.log p.Q + b * Real.log p.Q := by
41      calc
42        Real.log p.Q = (a + b) * Real.log p.Q := by rw [hab]; ring
43        _ = a * Real.log p.Q + b * Real.log p.Q := by ring
44    conv_lhs => rw [hconst]
45    ring
46
47  /-- Per-path value is convex on all hazard rates. -/
48  theorem value_convexOn (p : PathData) : ConvexOn ℝ univ p.value := by
49    constructor
50    · exact convex_univ
51    · intro x _hx y _hy a b ha hb hab
52      simpa [smul_eq_mul] using le_of_eq (value_affine p x y a b ha hb hab)
53
54  /-- Per-path value is concave on all hazard rates. -/
55  theorem value_concaveOn (p : PathData) : ConcaveOn ℝ univ p.value := by
56    constructor
57    · exact convex_univ
58    · intro x _hx y _hy a b ha hb hab
59      simpa [smul_eq_mul] using le_of_eq (value_affine p x y a b ha hb hab).symm
60
61  /-- Path action and path value are negatives of each other. -/
62  theorem action_neg_value (p : PathData) (lam : ℝ) :
63      p.action lam = -(p.value lam) := by
64    unfold PathData.action PathData.value
65    ring
66
67  end TINProofs.C1
68
69  end
```

TINProofs/C1/SupportConcave.lean

```
1   import TINProofs.C1.ActionAffine
2
3   open Real Set
4
5   noncomputable section
6
7   namespace TINProofs.C1
8
9   private theorem value_eq_neg_action (p : PathData) (lam : ℝ) :
10      p.value lam = -(p.action lam) := by
11    unfold PathData.action PathData.value
12    ring
13
14  /-- The support function is concave: finite infimum of affine actions. -/
15  theorem support_concaveOn (ch : CommodityHull) :
16      ConcaveOn ℝ univ ch.support := by
17    constructor
18    · exact convex_univ
19    · intro x _hx y _hy a b ha hb hab
20      unfold CommodityHull.support
21      simp only [smul_eq_mul]
22      apply Finset.le_inf'
23      intro p hp
24      have hx : ch.paths.inf' ch.hnonempty (fun q => q.action x) ≤ p.action x :=
25        Finset.inf'_le (s := ch.paths) (f := fun q => q.action x) hp
26      have hy : ch.paths.inf' ch.hnonempty (fun q => q.action y) ≤ p.action y :=
27        Finset.inf'_le (s := ch.paths) (f := fun q => q.action y) hp
28      calc
29        a * ch.paths.inf' ch.hnonempty (fun q => q.action x) +
30            b * ch.paths.inf' ch.hnonempty (fun q => q.action y)
31            ≤ a * p.action x + b * p.action y := by
32              exact add_le_add (mul_le_mul_of_nonneg_left hx ha)
33                (mul_le_mul_of_nonneg_left hy hb)
34        _ = p.action (a * x + b * y) := (action_affine p x y a b ha hb hab).symm
35
36  /-- The value function is the negation of the support function. -/
37  theorem valueFn_eq_neg_support (ch : CommodityHull) (lam : ℝ) :
38      ch.valueFn lam = -(ch.support lam) := by
39    unfold CommodityHull.valueFn CommodityHull.support
40    apply le_antisymm
41    · apply Finset.sup'_le
42      intro p hp
43      have hInf : ch.paths.inf' ch.hnonempty (fun q => q.action lam) ≤ p.action lam :=
```

```
      Finset.inf'_le (s := ch.paths) (f := fun q => q.action lam) hp
    calc
      p.value lam = -p.action lam := value_eq_neg_action p lam
      _ ≤ -(ch.paths.inf' ch.hnonempty (fun q => q.action lam)) := neg_le_neg hInf
  · obtain ⟨p, hp, hp_le⟩ :=
      (Finset.inf'_le_iff (H := ch.hnonempty) (f := fun q => q.action lam)
        (a := ch.paths.inf' ch.hnonempty (fun q => q.action lam))).mp le_rfl
    have hToValue :
        -(ch.paths.inf' ch.hnonempty (fun q => q.action lam)) ≤ p.value lam := by
      calc
        -(ch.paths.inf' ch.hnonempty (fun q => q.action lam)) ≤ -p.action lam :=
          neg_le_neg hp_le
        _ = p.value lam := (value_eq_neg_action p lam).symm
    exact hToValue.trans (Finset.le_sup' (s := ch.paths) (f := fun q => q.value lam) hp)

/-- The value function is convex: finite supremum of affine values. -/
theorem valueFn_convexOn (ch : CommodityHull) :
    ConvexOn ℝ univ ch.valueFn := by
  constructor
  · exact convex_univ
  · intro x _hx y _hy a b ha hb hab
    unfold CommodityHull.valueFn
    simp only [smul_eq_mul]
    apply Finset.sup'_le
    intro p hp
    have hx : p.value x ≤ ch.paths.sup' ch.hnonempty (fun q => q.value x) :=
      Finset.le_sup' (s := ch.paths) (f := fun q => q.value x) hp
    have hy : p.value y ≤ ch.paths.sup' ch.hnonempty (fun q => q.value y) :=
      Finset.le_sup' (s := ch.paths) (f := fun q => q.value y) hp
    calc
      p.value (a * x + b * y) = a * p.value x + b * p.value y :=
        value_affine p x y a b ha hb hab
      _ ≤ a * ch.paths.sup' ch.hnonempty (fun q => q.value x) +
          b * ch.paths.sup' ch.hnonempty (fun q => q.value y) := by
            exact add_le_add (mul_le_mul_of_nonneg_left hx ha)
              (mul_le_mul_of_nonneg_left hy hb)

end TINProofs.C1

end
```

TINProofs/C1/Crossover.lean

```
import TINProofs.C1.SupportConcave

open Real Set

noncomputable section

namespace TINProofs.C1

/-- At the crossover slope, two paths have equal action. -/
theorem action_eq_at_crossover (pi pj : PathData) (hT : pi.T ≠ pj.T) :
    pi.action (crossoverSlope pi pj hT) = pj.action (crossoverSlope pi pj hT) := by
  unfold PathData.action crossoverSlope
  have hden : pj.T - pi.T ≠ 0 := sub_ne_zero.mpr hT.symm
  field_simp [hden]
  ring

/-- Below the crossover slope, the higher-exposure path has lower action. -/
theorem action_lt_below_crossover (pi pj : PathData) (hT : pi.T < pj.T)
    (lam : ℝ) (hlam : lam < crossoverSlope pi pj (ne_of_lt hT)) :
    pj.action lam < pi.action lam := by
  unfold PathData.action crossoverSlope at *
  have hden_pos : 0 < pj.T - pi.T := sub_pos.mpr hT
  have hmul : lam * (pj.T - pi.T) < Real.log pj.Q - Real.log pi.Q :=
    (lt_div_iff₀ hden_pos).mp hlam
  nlinarith

/-- Above the crossover slope, the lower-exposure path has lower action. -/
theorem action_gt_above_crossover (pi pj : PathData) (hT : pi.T < pj.T)
    (lam : ℝ) (hlam : crossoverSlope pi pj (ne_of_lt hT) < lam) :
    pi.action lam < pj.action lam := by
  unfold PathData.action crossoverSlope at *
  have hden_pos : 0 < pj.T - pi.T := sub_pos.mpr hT
  have hmul : Real.log pj.Q - Real.log pi.Q < lam * (pj.T - pi.T) :=
    (div_lt_iff₀ hden_pos).mp hlam
  nlinarith

/-- Finite-path Legendre duality as the definitional relationship `V = -F`. -/
theorem legendre_duality (ch : CommodityHull) (lam : ℝ) :
    ch.valueFn lam = -(ch.support lam) :=
  valueFn_eq_neg_support ch lam

end TINProofs.C1

end
```

Appendix C2: Stochastic Certification Asymmetry

TINProofs/C2/Defs.lean

```
/-
  C2: Stochastic Certification Asymmetry — Definitions

  Formalizes the tier system and Binomial delivery ratio
  from Theorem S2 (stochastic_certification.tex).
-/
import Mathlib

open Finset BigOperators Nat Real

noncomputable section

/-- A tier classification system with M tiers and M-1 thresholds. -/
structure TierSystem where
  M : ℕ
  hM : 2 ≤ M
  k : ℕ
  hk : 0 < k
  τ : Fin (M - 1) → ℝ
  hτ_pos : ∀ i, 0 < τ i
  hτ_lt_one : ∀ i, τ i < 1
  hτ_strict_anti : StrictAnti τ

/-- Delivery ratio: DR = min(X/k, 1). -/
def deliveryRatio (k : ℕ) (X : ℕ) : ℝ :=
  min ((X : ℝ) / (k : ℝ)) 1

/-- Minimum integer X that achieves tier i: x_i = ⌈k · τ_i⌉. -/
def TierSystem.xMin (ts : TierSystem) (i : Fin (ts.M - 1)) : ℕ :=
  ⌈(ts.k : ℝ) * ts.τ i⌉₊

/-- Binomial probability mass function: P(X = j) for X ~ Binom(n, p). -/
def binomialPMF (n : ℕ) (p : ℝ) (j : ℕ) : ℝ :=
  (n.choose j : ℝ) * p ^ j * (1 - p) ^ (n - j)

/-- Binomial CDF: P(X ≤ m) for X ~ Binom(n, p). -/
def binomialCDF (n : ℕ) (p : ℝ) (m : ℕ) : ℝ :=
  ∑ j ∈ range (m + 1), binomialPMF n p j

/-- Upper tail: P(X ≥ m) for X ~ Binom(n, p). -/
def binomialUpperTail (n : ℕ) (p : ℝ) (m : ℕ) : ℝ :=
  ∑ j ∈ Icc m n, binomialPMF n p j

/-- The asymmetry ratio R_T: ratio of downgrade to upgrade probability.
    R_T = P(X ≤ x_T - 1) / P(X ≥ x_{T-1}) -/
def asymmetryRatio (n : ℕ) (p : ℝ) (xLower xUpper : ℕ) : ℝ :=
  binomialCDF n p (xLower - 1) / binomialUpperTail n p xUpper

/-- Poisson PMF: P(X = j) for X ~ Pois(mu). -/
def poissonPMF (mu : ℝ) (j : ℕ) : ℝ :=
  Real.exp (-mu) * mu ^ j / (j.factorial : ℝ)

/-- The set of achievable DR values: {0, 1/k, 2/k, ..., 1}. -/
def achievableDR (k : ℕ) : Finset ℝ :=
  (range (k + 1)).image (fun (i : ℕ) => (i : ℝ) / (k : ℝ))

end
```

TINProofs/C2/GapTiers.lean

```
/-
  C2 Part (v): Gap Tiers

  Tier T is unreachable when ⌈k·τ_T⌉ = ⌈k·τ_{T-1}⌉.
  The achievable DR values are {0, 1/k, ..., 1}; if no
  element falls in [τ_T, τ_{T-1}), the tier is empty.
-/
import TINProofs.C2.Defs

open Finset BigOperators Nat

/-- No integer X can satisfy xLower ≤ X < xUpper when xLower = xUpper. -/
theorem no_achievable_in_gap (xLower xUpper : ℕ) (h : xLower = xUpper) :
    ∀ X : ℕ, ¬(xLower ≤ X ∧ X < xUpper) := by
  intro X ⟨h1, h2⟩
  omega

/-- Gap tier theorem: tier T is unreachable when the ceiling boundaries
    coincide. This is Part (v) of Theorem S2. -/
theorem gap_tier (ts : TierSystem)
    (i j : Fin (ts.M - 1))
    (h_gap : ts.xMin i = ts.xMin j) :
```

```
    ∀ X : ℕ, ¬(ts.xMin i ≤ X ∧ X < ts.xMin j) :=
  no_achievable_in_gap _ _ h_gap
```

TINProofs/C2/AsymmetryRatio.lean

```
/-
  C2 Part (ii): Asymmetry Ratio

  The tier asymmetry ratio is the quotient of two finite Binomial sums
  determined by n, p, k, and the tier thresholds.
-/
import TINProofs.C2.Defs

open Finset BigOperators Nat

noncomputable section

/-- Tier cutoffs are positive because k > 0 and every threshold is positive. -/
theorem TierSystem.xMin_pos (ts : TierSystem) (i : Fin (ts.M - 1)) :
    0 < ts.xMin i := by
  unfold TierSystem.xMin
  rw [Nat.ceil_pos]
  exact mul_pos (by exact_mod_cast ts.hk) (ts.hτ_pos i)

/-- For a positive cutoff x, the CDF at x - 1 is the finite lower-tail sum
    over indices 0, ..., x - 1. -/
theorem binomialCDF_pred_eq_sum_range (n : ℕ) (p : ℝ) (x : ℕ) (hx : 0 < x) :
    binomialCDF n p (x - 1) = ∑ j ∈ range x, binomialPMF n p j := by
  unfold binomialCDF
  rw [Nat.sub_one_add_one_eq_of_pos hx]

/-- The tier-specific asymmetry ratio obtained from the two tier cutoffs. -/
def tierAsymmetryRatio (ts : TierSystem) (n : ℕ) (p : ℝ)
    (T Tpred : Fin (ts.M - 1)) : ℝ :=
  asymmetryRatio n p (ts.xMin T) (ts.xMin Tpred)

/-- Part (ii): the asymmetry ratio is exactly the quotient of finite
    Binomial lower-tail and upper-tail sums. -/
theorem tierAsymmetryRatio_eq_binomial_sums (ts : TierSystem) (n : ℕ) (p : ℝ)
    (T Tpred : Fin (ts.M - 1)) :
    tierAsymmetryRatio ts n p T Tpred =
      (∑ j ∈ range (ts.xMin T), binomialPMF n p j) /
        (∑ j ∈ Icc (ts.xMin Tpred) n, binomialPMF n p j) := by
  unfold tierAsymmetryRatio asymmetryRatio binomialUpperTail
  rw [binomialCDF_pred_eq_sum_range n p (ts.xMin T) (ts.xMin_pos T)]

end
```

TINProofs/C2/Absorbing.lean

```
/-
  C2 Part (i): Absorbing Lowest Tier

  This file proves the source-controlled cutoff arithmetic behind the
  absorbing-tier claim. The Binomial median and mode facts are kept as
  explicit hypotheses; the project does not yet define Binomial medians
  or modes as first-class objects.
-/
import TINProofs.C2.Defs

open Finset BigOperators Nat

noncomputable section

/-- The last threshold, corresponding to the boundary above the lowest tier. -/
def TierSystem.lowestThreshold (ts : TierSystem) : Fin (ts.M - 1) :=
  ⟨ts.M - 2, by
    have hM : 2 ≤ ts.M := ts.hM
    omega⟩

/-- If the Binomial mean lies more than one unit below a tier cutoff in real
    coordinates, then its natural ceiling lies strictly below the integer
    cutoff. -/
theorem ceil_mean_lt_xMin_of_lt_sub_one (ts : TierSystem) (i : Fin (ts.M - 1))
    (n : ℕ) (p : ℝ) (hp : 0 ≤ p)
    (h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
    ⌈(n : ℝ) * p⌉₊ < ts.xMin i := by
  unfold TierSystem.xMin
  rw [Nat.lt_ceil]
  have hmean_nonneg : 0 ≤ (n : ℝ) * p := by
    exact mul_nonneg (by exact_mod_cast Nat.zero_le n) hp
  have hceil : (⌈(n : ℝ) * p⌉₊ : ℝ) < (n : ℝ) * p + 1 :=
    Nat.ceil_lt_add_one hmean_nonneg
  linarith

/-- Non-strict form matching the displayed source inequality. -/
theorem ceil_mean_le_xMin_of_lt_sub_one (ts : TierSystem) (i : Fin (ts.M - 1))
    (n : ℕ) (p : ℝ) (hp : 0 ≤ p)
```

```
    (h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
    ⌈(n : ℝ) * p⌉₊ ≤ ts.xMin i :=
  le_of_lt (ceil_mean_lt_xMin_of_lt_sub_one ts i n p hp h)

/-- Conditional median consequence: once the standard Binomial median upper
    bound is supplied, the median is below the tier cutoff. -/
theorem median_lt_xMin_of_median_le_ceil_mean (ts : TierSystem)
    (i : Fin (ts.M - 1)) (n median : ℕ) (p : ℝ) (hp : 0 ≤ p)
    (hmedian : median ≤ ⌈(n : ℝ) * p⌉₊)
    (h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
    median < ts.xMin i :=
  lt_of_le_of_lt hmedian (ceil_mean_lt_xMin_of_lt_sub_one ts i n p hp h)

/-- Conditional mode consequence: if the mode is no larger than such a median,
    then the mode is also below the tier cutoff. -/
theorem mode_lt_xMin_of_mode_le_median (ts : TierSystem)
    (i : Fin (ts.M - 1)) (n mode median : ℕ) (p : ℝ) (hp : 0 ≤ p)
    (hmode : mode ≤ median)
    (hmedian : median ≤ ⌈(n : ℝ) * p⌉₊)
    (h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ i - 1) :
    mode < ts.xMin i :=
  lt_of_le_of_lt hmode
    (median_lt_xMin_of_median_le_ceil_mean ts i n median p hp hmedian h)

/-- Lowest-tier specialization of the ceiling cutoff bound. -/
theorem absorbing_lowest_tier_ceiling_lt_cutoff (ts : TierSystem)
    (n : ℕ) (p : ℝ) (hp : 0 ≤ p)
    (h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ ts.lowestThreshold - 1) :
    ⌈(n : ℝ) * p⌉₊ < ts.xMin ts.lowestThreshold :=
  ceil_mean_lt_xMin_of_lt_sub_one ts ts.lowestThreshold n p hp h

/-- Lowest-tier specialization with the Binomial median upper bound supplied
    as an explicit hypothesis. -/
theorem absorbing_lowest_tier_median_lt_cutoff (ts : TierSystem)
    (n median : ℕ) (p : ℝ) (hp : 0 ≤ p)
    (hmedian : median ≤ ⌈(n : ℝ) * p⌉₊)
    (h : (n : ℝ) * p < (ts.k : ℝ) * ts.τ ts.lowestThreshold - 1) :
    median < ts.xMin ts.lowestThreshold :=
  median_lt_xMin_of_median_le_ceil_mean
    ts ts.lowestThreshold n median p hp hmedian h

end
```

TINProofs/C2/PoissonLimit.lean

```
/-
  C2 Part (iii): Poisson Limit

  This file pins down the Poisson crossover equation that the
  Binomial-to-Poisson limit must converge to.
-/
import TINProofs.C2.Defs
import Mathlib.Probability.Distributions.Poisson.PoissonLimitThm

open Finset BigOperators Nat
open scoped BigOperators
open Filter Topology

noncomputable section

/-- Lower Poisson tail through m. -/
def poissonLowerTail (mu : ℝ) (m : ℕ) : ℝ :=
  ∑ j ∈ range (m + 1), poissonPMF mu j

/-- Upper Poisson tail from k onward. -/
def poissonUpperTail (mu : ℝ) (k : ℕ) : ℝ :=
  ∑' j : ℕ, if k ≤ j then poissonPMF mu j else 0

/-- Poisson crossover equation from the source theorem:
    P(X <= k - 2; mu) = P(X >= k; mu). -/
def poissonCrossoverCondition (k : ℕ) (mu : ℝ) : Prop :=
  poissonLowerTail mu (k - 2) = poissonUpperTail mu k

/-- Expanded form of the Poisson crossover condition. -/
theorem poissonCrossoverCondition_iff (k : ℕ) (mu : ℝ) :
    poissonCrossoverCondition k mu ↔
      (∑ j ∈ range (k - 2 + 1), poissonPMF mu j) =
        (∑' j : ℕ, if k ≤ j then poissonPMF mu j else 0) := by
  rfl

/-- Pointwise Binomial-to-Poisson convergence for the local real-valued PMF
    definitions. This is the law-of-small-numbers input for finite-tail
    convergence. -/
theorem binomialPMF_tendsto_poissonPMF (k : ℕ) {r : ℝ} {p : ℕ → ℝ}
    (hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
    Tendsto (fun n => binomialPMF n (p n) k) atTop (𝓝 (poissonPMF r k)) := by
  simpa [binomialPMF, poissonPMF] using
    ProbabilityTheory.tendsto_choose_mul_pow_of_tendsto_mul_atTop (p := p) (r := r) k hr

```

```
/-- Fixed finite lower tails converge termwise under the
    Binomial-to-Poisson scaling. -/
theorem binomialLowerTail_tendsto_poissonLowerTail (m : ℕ) {r : ℝ} {p : ℕ → ℝ}
    (hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
    Tendsto (fun n => ∑ j ∈ range (m + 1), binomialPMF n (p n) j)
      atTop (𝓝 (poissonLowerTail r m)) := by
  simpa [poissonLowerTail] using
    tendsto_finset_sum (range (m + 1))
      (fun j _hj => binomialPMF_tendsto_poissonPMF j hr)

/-- The local real-valued Binomial PMF sums to one over its finite support. -/
theorem binomialPMF_sum_range_eq_one (n : ℕ) (p : ℝ) :
    ∑ j ∈ range (n + 1), binomialPMF n p j = 1 := by
  have h := add_pow p (1 - p) n
  have hp : p + (1 - p) = (1 : ℝ) := by ring
  rw [hp, one_pow] at h
  rw [h]
  simp [binomialPMF, mul_assoc, mul_left_comm, mul_comm]

/-- Natural closed intervals as half-open intervals with a successor endpoint. -/
lemma Icc_eq_Ico_succ_nat (k n : ℕ) : Icc k n = Ico k (n + 1) := by
  ext j
  simp

/-- Binomial upper tails are complements of finite lower tails once the lower
    cutoff lies inside the finite support. -/
theorem binomialUpperTail_eq_one_sub_lower (n : ℕ) (p : ℝ) (k : ℕ)
    (hk : k ≤ n + 1) :
    binomialUpperTail n p k = 1 - ∑ j ∈ range k, binomialPMF n p j := by
  unfold binomialUpperTail
  rw [Icc_eq_Ico_succ_nat]
  have hsplit := sum_range_add_sum_Ico (fun j => binomialPMF n p j) hk
  have htotal := binomialPMF_sum_range_eq_one n p
  linarith

/-- The local real-valued Poisson PMF sums to one. -/
theorem hasSum_poissonPMF (mu : ℝ) : HasSum (fun j : ℕ => poissonPMF mu j) 1 := by
  have hs : HasSum (fun j : ℕ => mu ^ j / (j.factorial : ℝ)) (Real.exp mu) := by
    simpa [Real.exp_eq_exp_ℝ] using (NormedSpace.expSeries_div_hasSum_exp mu)
  have hs' := hs.mul_left (Real.exp (-mu))
  have hone : Real.exp (-mu) * Real.exp mu = 1 := by
    rw [← Real.exp_add]
    ring_nf
    simp
  rw [← hone]
  simpa [poissonPMF, mul_div_assoc] using hs'

/-- Reindex a tail over `{j | k <= j}` as a sum over `i + k`. -/
theorem tsum_ite_ge_eq_nat_add (f : ℕ → ℝ) (k : ℕ) :
    (∑' j : ℕ, if k ≤ j then f j else 0) = ∑' i : ℕ, f (i + k) := by
  have hinj : Function.Injective (fun i : ℕ => i + k) := by
    intro a b h
    exact Nat.add_right_cancel h
  have hsupport : Function.support (fun j : ℕ => if k ≤ j then f j else 0) ⊆
      Set.range (fun i : ℕ => i + k) := by
    intro j hj
    simp only [Function.mem_support] at hj
    by_cases hkj : k ≤ j
    · exact ⟨j - k, Nat.sub_add_cancel hkj⟩
    · simp [hkj] at hj
  have h := hinj.tsum_eq (f := fun j : ℕ => if k ≤ j then f j else 0) hsupport
  rw [← h]
  simp

/-- Poisson upper tails are complements of finite lower tails. -/
theorem poissonUpperTail_eq_one_sub_lower (mu : ℝ) (k : ℕ) :
    poissonUpperTail mu k = 1 - ∑ j ∈ range k, poissonPMF mu j := by
  unfold poissonUpperTail
  rw [tsum_ite_ge_eq_nat_add]
  have hs := hasSum_poissonPMF mu
  have hsplit := Summable.sum_add_tsum_nat_add k hs.summable
  have htotal := hs.tsum_eq
  linarith

/-- The zero-rate Poisson mass at zero. -/
theorem poissonPMF_zero_zero : poissonPMF 0 0 = 1 := by
  simp [poissonPMF]

/-- The zero-rate Poisson mass away from zero. -/
theorem poissonPMF_zero_of_pos (j : ℕ) (hj : 0 < j) : poissonPMF 0 j = 0 := by
  simp [poissonPMF, hj.ne']

/-- Any positive finite lower range contains the full zero-rate Poisson mass. -/
theorem sum_range_poissonPMF_zero_eq_one (k : ℕ) (hk : 0 < k) :
    ∑ j ∈ range k, poissonPMF 0 j = 1 := by
  rw [sum_eq_single 0]
  · exact poissonPMF_zero_zero
  · intro j hj hne
    apply poissonPMF_zero_of_pos
    omega
```

```
  · intro h0
    simp [hk] at h0

/-- At zero rate, every positive upper tail is zero. -/
theorem poissonUpperTail_zero_of_pos (k : ℕ) (hk : 0 < k) :
    poissonUpperTail 0 k = 0 := by
  rw [poissonUpperTail_eq_one_sub_lower, sum_range_poissonPMF_zero_eq_one k hk]
  norm_num

/-- At zero rate, every finite lower tail is one. -/
theorem poissonLowerTail_zero_eq_one (m : ℕ) :
    poissonLowerTail 0 m = 1 := by
  unfold poissonLowerTail
  apply sum_range_poissonPMF_zero_eq_one
  omega

/-- Fixed finite lower tails written with an exact `range k` cutoff converge
    termwise under the Binomial-to-Poisson scaling. -/
theorem binomialFiniteLowerTail_tendsto_poissonFiniteLowerTail
    (k : ℕ) {r : ℝ} {p : ℕ → ℝ}
    (hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
    Tendsto (fun n => ∑ j ∈ range k, binomialPMF n (p n) j)
      atTop (𝓝 (∑ j ∈ range k, poissonPMF r j)) := by
  exact tendsto_finset_sum (range k)
    (fun j _hj => binomialPMF_tendsto_poissonPMF j hr)

/-- Upper-tail convergence under the Binomial-to-Poisson scaling. -/
theorem binomialUpperTail_tendsto_poissonUpperTail
    (k : ℕ) {r : ℝ} {p : ℕ → ℝ}
    (hr : Tendsto (fun n => n * p n) atTop (𝓝 r)) :
    Tendsto (fun n => binomialUpperTail n (p n) k) atTop (𝓝 (poissonUpperTail r k)) := by
  have hEq : (fun n => binomialUpperTail n (p n) k) =ᶠ[atTop]
      (fun n => 1 - ∑ j ∈ range k, binomialPMF n (p n) j) := by
    filter_upwards [eventually_ge_atTop (k - 1)] with n hn
    rw [binomialUpperTail_eq_one_sub_lower]
    omega
  refine Tendsto.congr' hEq.symm ?_
  have hlim : Tendsto (fun n => (1 : ℝ) - ∑ j ∈ range k, binomialPMF n (p n) j)
      atTop (𝓝 ((1 : ℝ) - ∑ j ∈ range k, poissonPMF r j)) :=
    (tendsto_const_nhds (x := (1 : ℝ))).sub
      (binomialFiniteLowerTail_tendsto_poissonFiniteLowerTail k hr)
  simpa [poissonUpperTail_eq_one_sub_lower] using hlim

/-- Continuity of each local Poisson PMF term. -/
theorem continuous_poissonPMF (j : ℕ) : Continuous fun mu : ℝ => poissonPMF mu j := by
  unfold poissonPMF
  fun_prop

/-- Continuity of finite Poisson lower tails. -/
theorem continuous_poissonLowerTail (m : ℕ) : Continuous fun mu : ℝ => poissonLowerTail mu m := by
  unfold poissonLowerTail
  exact continuous_finset_sum (range (m + 1)) (fun j _hj => continuous_poissonPMF j)

/-- Continuity of finite lower-tail sums with exact `range k` cutoffs. -/
theorem continuous_poissonFiniteLowerTail (k : ℕ) :
    Continuous fun mu : ℝ => ∑ j ∈ range k, poissonPMF mu j := by
  exact continuous_finset_sum (range k) (fun j _hj => continuous_poissonPMF j)

/-- Continuity of Poisson upper tails, via the finite complement identity. -/
theorem continuous_poissonUpperTail (k : ℕ) :
    Continuous fun mu : ℝ => poissonUpperTail mu k := by
  have hfun : (fun mu : ℝ => poissonUpperTail mu k) =
      fun mu : ℝ => 1 - ∑ j ∈ range k, poissonPMF mu j := by
    funext mu
    rw [poissonUpperTail_eq_one_sub_lower]
  rw [hfun]
  exact continuous_const.sub (continuous_poissonFiniteLowerTail k)

/-- IVT root-crossing theorem for any explicit bracket whose endpoints put
    the lower and upper Poisson tails on opposite sides. -/
theorem poissonCrossover_exists_of_bracket (k : ℕ) (a b : ℝ) (hab : a ≤ b)
    (ha : poissonUpperTail a k ≤ poissonLowerTail a (k - 2))
    (hb : poissonLowerTail b (k - 2) ≤ poissonUpperTail b k) :
    ∃ mu ∈ Set.Icc a b, poissonCrossoverCondition k mu := by
  have hcontU : ContinuousOn (fun mu : ℝ => poissonUpperTail mu k) (Set.Icc a b) :=
    (continuous_poissonUpperTail k).continuousOn
  have hcontL : ContinuousOn (fun mu : ℝ => poissonLowerTail mu (k - 2)) (Set.Icc a b) :=
    (continuous_poissonLowerTail (k - 2)).continuousOn
  obtain ⟨mu, hmu, hEq⟩ := IsPreconnected.intermediate_value₂
    (s := Set.Icc a b) isPreconnected_Icc
    (a := a) (b := b)
    (f := fun mu : ℝ => poissonUpperTail mu k)
    (g := fun mu : ℝ => poissonLowerTail mu (k - 2))
    (by simp [hab]) (by simp [hab]) hcontU hcontL ha hb
  exact ⟨mu, hmu, hEq.symm⟩

/-- Root-crossing existence from the zero-rate endpoint and any finite right
    endpoint where the Poisson tail inequality has reversed. -/
theorem poissonCrossover_exists_of_reverse_at (k : ℕ) (b : ℝ)
    (hk : 2 ≤ k) (hb0 : 0 ≤ b)
```

```
    (hb : poissonLowerTail b (k - 2) ≤ poissonUpperTail b k) :
    ∃ mu ∈ Set.Icc 0 b, poissonCrossoverCondition k mu := by
  apply poissonCrossover_exists_of_bracket k 0 b hb0
  · rw [poissonUpperTail_zero_of_pos k (by omega), poissonLowerTail_zero_eq_one (k - 2)]
    norm_num
  · exact hb

end
```

TINProofs/C2/Expansion.lean

```
/-
  C2 Part (iv): Asymptotic Expansion

  The external Poisson-median asymptotic input is kept as a named
  hypothesis. This file proves the algebraic step from the lambda-star
  expansion to the delivery-ratio expansion.
-/
import TINProofs.C2.PoissonLimit

open Filter Topology Asymptotics

noncomputable section

/-- The asymptotic inverse of the reserve-cycle count. -/
def invK (k : ℕ) : ℝ :=
  ((k : ℝ)⁻¹)

/-- The `1 / k^2` scale used for the delivery-ratio remainder. -/
def invKSq (k : ℕ) : ℝ :=
  invK k ^ 2

/-- Center term for the Poisson crossover expansion:
    `lambda* = k - 5/6 + O(1/k)`. -/
def lambdaCenter (k : ℕ) : ℝ :=
  (k : ℝ) - (5 / 6 : ℝ)

/-- Delivery ratio induced by a selected Poisson crossover sequence. -/
def drStar (lambdaStar : ℕ → ℝ) (k : ℕ) : ℝ :=
  lambdaStar k / (k : ℝ)

/-- Target delivery-ratio expansion center: `1 - 5/(6k)`. -/
def drTarget (k : ℕ) : ℝ :=
  1 - (5 / 6 : ℝ) / (k : ℝ)

/-- A sequence selects Poisson crossover roots for all sufficiently large k. -/
def selectsPoissonCrossover (lambdaStar : ℕ → ℝ) : Prop :=
  ∀ᶠ k in atTop, poissonCrossoverCondition k (lambdaStar k)

/-- Named external input corresponding to the Poisson-median asymptotics:
    `lambda* = k - 5/6 + O(1/k)`. -/
def adellJodraAsymptoticInput (lambdaStar : ℕ → ℝ) : Prop :=
  (fun k : ℕ => lambdaStar k - lambdaCenter k) =O[atTop] invK

/-- Part (iv), conditional on the external Poisson-median asymptotic input:
    dividing `lambda* = k - 5/6 + O(1/k)` by k gives
    `DR* = 1 - 5/(6k) + O(1/k^2)`. -/
theorem drStar_expansion_of_lambda_expansion (lambdaStar : ℕ → ℝ)
    (_hCross : selectsPoissonCrossover lambdaStar)
    (hLam : adellJodraAsymptoticInput lambdaStar) :
    (fun k : ℕ => drStar lambdaStar k - drTarget k) =O[atTop] invKSq := by
  have hmul : (fun k : ℕ => (lambdaStar k - lambdaCenter k) * invK k) =O[atTop]
      (fun k : ℕ => invK k * invK k) :=
    hLam.mul (isBigO_refl invK atTop)
  refine hmul.congr' ?_ ?_
  · filter_upwards [eventually_ge_atTop 1] with k hk
    unfold drStar drTarget lambdaCenter invK
    have hk0 : (k : ℝ) ≠ 0 := by
      exact_mod_cast (Nat.ne_of_gt hk)
    field_simp [hk0]
  · exact Eventually.of_forall (fun k => by simp [invKSq, pow_two])

end
```

Appendix C3: Lyapunov Radius of Validity

TINProofs/C3/Defs.lean

```
/-
  C3: Lyapunov Radius of Validity -- definitions

  This file keeps the linear-algebra inputs as explicit real inequalities.
  The later files prove the radius-of-validity algebra from those inputs.
-/
import Mathlib

noncomputable section

namespace TINProofs.C3

/-- Scalar data used by the Lyapunov radius estimate. -/
structure LyapunovSetup where
  alphaMin : ℝ
  alphaMax : ℝ
  q : ℝ
  r0 : ℝ
  opNormA : ℝ
  M : ℝ
  hAlphaMin_pos : 0 < alphaMin
  hAlphaMax_pos : 0 < alphaMax
  hq_pos : 0 < q
  hr0_pos : 0 < r0
  hOpNormA_pos : 0 < opNormA
  hM_pos : 0 < M

/-- Critical radius `r* = min(r0, q / (2 ||A|| M))`. -/
def criticalRadius (S : LyapunovSetup) : ℝ :=
  min S.r0 (S.q / (2 * S.opNormA * S.M))

/-- Exponential decay rate in the Lyapunov estimate. -/
def lyapunovDecayRate (S : LyapunovSetup) : ℝ :=
  S.q / (2 * S.alphaMax)

/--
Pointwise scalar facts for one perturbation.

`rho` is `‖delta‖`, `V` is the quadratic Lyapunov value, `quad` is
`delta^T Q delta`, and `remainder` is `2 delta^T A R(delta)`.
-/
structure LyapunovPoint (S : LyapunovSetup) where
  rho : ℝ
  V : ℝ
  Vdot : ℝ
  quad : ℝ
  remainder : ℝ
  hrho_nonneg : 0 ≤ rho
  eigenSandwich : S.alphaMin * rho ^ 2 ≤ V ∧ V ≤ S.alphaMax * rho ^ 2
  vdotDecomp : Vdot = -quad + remainder
  quadLower : S.q * rho ^ 2 ≤ quad
  remainderAbsBound : |remainder| ≤ S.opNormA * S.M * rho ^ 3

end TINProofs.C3

end
```

TINProofs/C3/EigenSandwich.lean

```
/-
  C3 Lemma 1: eigenvalue sandwich for the quadratic Lyapunov value.
-/
import TINProofs.C3.Defs

noncomputable section

namespace TINProofs.C3

variable {S : LyapunovSetup} (P : LyapunovPoint S)

/-- Lower Rayleigh-quotient bound for the Lyapunov quadratic form. -/
theorem eigen_sandwich_lower :
    S.alphaMin * P.rho ^ 2 ≤ P.V :=
  P.eigenSandwich.1

/-- Upper Rayleigh-quotient bound for the Lyapunov quadratic form. -/
theorem eigen_sandwich_upper :
    P.V ≤ S.alphaMax * P.rho ^ 2 :=
  P.eigenSandwich.2

/-- The full Lyapunov eigenvalue sandwich. -/
theorem eigen_sandwich :
```

```
    S.alphaMin * P.rho ^ 2 ≤ P.V ∧ P.V ≤ S.alphaMax * P.rho ^ 2 :=
  P.eigenSandwich

end TINProofs.C3

end
```

TINProofs/C3/VDotDecomp.lean

```
/-
  C3 Lemma 2: decomposition of the Lyapunov derivative.
-/
import TINProofs.C3.Defs

noncomputable section

namespace TINProofs.C3

variable {S : LyapunovSetup} (P : LyapunovPoint S)

/--
Pointwise derivative decomposition:
`Vdot = -delta^T Q delta + 2 delta^T A R(delta)`.
-/
theorem vdot_decomposition :
    P.Vdot = -P.quad + P.remainder :=
  P.vdotDecomp

end TINProofs.C3

end
```

TINProofs/C3/QuadBound.lean

```
/-
  C3 Lemma 3: lower bound for the stabilizing quadratic term.
-/
import TINProofs.C3.Defs

noncomputable section

namespace TINProofs.C3

variable {S : LyapunovSetup} (P : LyapunovPoint S)

/-- The Lyapunov matrix quadratic form is bounded below by `q * rho^2`. -/
theorem quadratic_term_lower :
    S.q * P.rho ^ 2 ≤ P.quad :=
  P.quadLower

/-- The negative quadratic term is bounded above by `-q * rho^2`. -/
theorem negative_quadratic_term_upper :
    -P.quad ≤ -(S.q * P.rho ^ 2) :=
  neg_le_neg P.quadLower

end TINProofs.C3

end
```

TINProofs/C3/RemainderBound.lean

```
/-
  C3 Lemma 4: Taylor remainder contribution bound.
-/
import TINProofs.C3.Defs

noncomputable section

namespace TINProofs.C3

variable {S : LyapunovSetup} (P : LyapunovPoint S)

/-- Absolute bound for the nonlinear Lyapunov remainder term. -/
theorem remainder_abs_bound :
    |P.remainder| ≤ S.opNormA * S.M * P.rho ^ 3 :=
  P.remainderAbsBound

/-- One-sided form of the nonlinear remainder bound. -/
theorem remainder_upper_bound :
    P.remainder ≤ S.opNormA * S.M * P.rho ^ 3 :=
  le_trans (le_abs_self P.remainder) P.remainderAbsBound

end TINProofs.C3

end
```

TINProofs/C3/BallBound.lean

```
/-
  C3 Lemma 5: derivative bound inside the critical ball.
-/
import TINProofs.C3.VDotDecomp
import TINProofs.C3.QuadBound
import TINProofs.C3.RemainderBound

noncomputable section

namespace TINProofs.C3

variable {S : LyapunovSetup} (P : LyapunovPoint S)

/-- Combining the decomposition, quadratic lower bound, and remainder bound. -/
theorem vdot_le_quadratic_plus_cubic :
    P.Vdot ≤ -S.q * P.rho ^ 2 + S.opNormA * S.M * P.rho ^ 3 := by
  calc
    P.Vdot = -P.quad + P.remainder := vdot_decomposition P
    _ ≤ -(S.q * P.rho ^ 2) + S.opNormA * S.M * P.rho ^ 3 := by
      exact add_le_add (negative_quadratic_term_upper P) (remainder_upper_bound P)
    _ = -S.q * P.rho ^ 2 + S.opNormA * S.M * P.rho ^ 3 := by
      ring

/-- On `rho < r*`, the nonlinear term consumes at most half the quadratic margin. -/
theorem vdot_bound_on_ball (hball : P.rho < criticalRadius S) :
    P.Vdot ≤ -(S.q / 2) * P.rho ^ 2 := by
  have hsmall : P.rho < S.q / (2 * S.opNormA * S.M) :=
    lt_of_lt_of_le hball (min_le_right S.r0 (S.q / (2 * S.opNormA * S.M)))
  have hden_pos : 0 < 2 * S.opNormA * S.M := by
    nlinarith [S.hOpNormA_pos, S.hM_pos]
  have hprod_lt : P.rho * (2 * S.opNormA * S.M) < S.q :=
    (lt_div_iff₀ hden_pos).mp hsmall
  have hcoeff_lt : S.opNormA * S.M * P.rho < S.q / 2 := by
    nlinarith
  have hsq_nonneg : 0 ≤ P.rho ^ 2 := sq_nonneg P.rho
  have hcubic :
      S.opNormA * S.M * P.rho ^ 3 ≤ (S.q / 2) * P.rho ^ 2 := by
    calc
      S.opNormA * S.M * P.rho ^ 3 =
          (S.opNormA * S.M * P.rho) * P.rho ^ 2 := by
        ring
      _ ≤ (S.q / 2) * P.rho ^ 2 := by
        exact mul_le_mul_of_nonneg_right (le_of_lt hcoeff_lt) hsq_nonneg
  calc
    P.Vdot ≤ -S.q * P.rho ^ 2 + S.opNormA * S.M * P.rho ^ 3 :=
      vdot_le_quadratic_plus_cubic P
    _ ≤ -S.q * P.rho ^ 2 + (S.q / 2) * P.rho ^ 2 := by
      nlinarith [hcubic]
    _ = -(S.q / 2) * P.rho ^ 2 := by
      ring

end TINProofs.C3

end
```

TINProofs/C3/RadiusOfValidity.lean

```
/-
  C3 main theorem: Lyapunov radius of validity.
-/
import TINProofs.C3.EigenSandwich
import TINProofs.C3.BallBound

noncomputable section

namespace TINProofs.C3

variable {S : LyapunovSetup} (P : LyapunovPoint S)

/-- The critical radius is positive under the setup hypotheses. -/
theorem criticalRadius_pos (S : LyapunovSetup) :
    0 < criticalRadius S := by
  have hden_pos : 0 < 2 * S.opNormA * S.M := by
    nlinarith [S.hOpNormA_pos, S.hM_pos]
  have hfrac_pos : 0 < S.q / (2 * S.opNormA * S.M) :=
    div_pos S.hq_pos hden_pos
  exact lt_min S.hr0_pos hfrac_pos

/--
Inside the critical ball, the Lyapunov derivative has the exponential
decay rate `q / (2 * alphaMax)`.
-/
theorem lyapunov_radius_of_validity (hball : P.rho < criticalRadius S) :
    P.Vdot ≤ -(lyapunovDecayRate S) * P.V := by
  have hball_bound : P.Vdot ≤ -(S.q / 2) * P.rho ^ 2 :=
    vdot_bound_on_ball P hball
```

```
  have hupper : P.V ≤ S.alphaMax * P.rho ^ 2 :=
    eigen_sandwich_upper P
  have hden_pos : 0 < 2 * S.alphaMax := by
    nlinarith [S.hAlphaMax_pos]
  have hrate_pos : 0 < S.q / (2 * S.alphaMax) :=
    div_pos S.hq_pos hden_pos
  have hcoef_nonpos : -(S.q / (2 * S.alphaMax)) ≤ 0 := by
    nlinarith
  have hscaled :
      -(S.q / (2 * S.alphaMax)) * (S.alphaMax * P.rho ^ 2) ≤
        -(S.q / (2 * S.alphaMax)) * P.V :=
    mul_le_mul_of_nonpos_left hupper hcoef_nonpos
  have hrewrite :
      -(S.q / 2) * P.rho ^ 2 =
        -(S.q / (2 * S.alphaMax)) * (S.alphaMax * P.rho ^ 2) := by
    field_simp [ne_of_gt S.hAlphaMax_pos]
  calc
    P.Vdot ≤ -(S.q / 2) * P.rho ^ 2 := hball_bound
    _ = -(S.q / (2 * S.alphaMax)) * (S.alphaMax * P.rho ^ 2) := hrewrite
    _ ≤ -(S.q / (2 * S.alphaMax)) * P.V := hscaled
    _ = -(lyapunovDecayRate S) * P.V := by
      rfl

/--
Any Lyapunov sublevel value below `alphaMin * r*^2` lies inside the
critical ball.
-/
theorem sublevel_inside_critical_ball {V0 : ℝ}
    (hV : P.V ≤ V0)
    (hV0 : V0 < S.alphaMin * criticalRadius S ^ 2) :
    P.rho < criticalRadius S := by
  have hchain :
      S.alphaMin * P.rho ^ 2 < S.alphaMin * criticalRadius S ^ 2 :=
    lt_of_le_of_lt (le_trans (eigen_sandwich_lower P) hV) hV0
  have hsquares : P.rho ^ 2 < criticalRadius S ^ 2 :=
    by nlinarith [S.hAlphaMin_pos, hchain]
  have hradius_pos : 0 < criticalRadius S := criticalRadius_pos S
  nlinarith [P.hrho_nonneg, hradius_pos, hsquares,
    sq_nonneg (P.rho - criticalRadius S)]

/--
Algebraic forward-invariance criterion: on a boundary value below
`alphaMin * r*^2`, the Lyapunov derivative is nonpositive.
-/
theorem sublevel_boundary_vdot_nonpos {V0 : ℝ}
    (hV : P.V = V0)
    (hV0 : V0 < S.alphaMin * criticalRadius S ^ 2) :
    P.Vdot ≤ 0 := by
  have hinside : P.rho < criticalRadius S :=
    sublevel_inside_critical_ball P (le_of_eq hV) hV0
  have hdecay : P.Vdot ≤ -(lyapunovDecayRate S) * P.V :=
    lyapunov_radius_of_validity P hinside
  have hV_nonneg : 0 ≤ P.V := by
    have hlow : S.alphaMin * P.rho ^ 2 ≤ P.V := eigen_sandwich_lower P
    have hrho_sq_nonneg : 0 ≤ P.rho ^ 2 := sq_nonneg P.rho
    nlinarith [S.hAlphaMin_pos, hlow, hrho_sq_nonneg]
  have hrate_pos : 0 < lyapunovDecayRate S := by
    unfold lyapunovDecayRate
    have hden_pos : 0 < 2 * S.alphaMax := by
      nlinarith [S.hAlphaMax_pos]
    exact div_pos S.hq_pos hden_pos
  have hright_nonpos : -(lyapunovDecayRate S) * P.V ≤ 0 := by
    nlinarith
  exact le_trans hdecay hright_nonpos

end TINProofs.C3

end
```

Appendix C4: Temporal Transport Factorization

TINProofs/C4/Defs.lean

```
/-
  C4: Temporal transport factorization -- definitions.

  The C4 layer works at the level of abstract probability events rather than
  concrete temporal graphs.
-/
import Mathlib

open MeasureTheory Set

noncomputable section

namespace TINProofs.C4

/--
A temporal transport instance: a probability space with delivery and
feasibility events.
-/
structure TemporalTransport (Ω : Type*) [MeasurableSpace Ω] where
  μ : Measure Ω
  hprob : IsProbabilityMeasure μ
  D : Set Ω
  F : Set Ω
  hD : MeasurableSet D
  hF : MeasurableSet F
  h_incl : D ⊆ F

namespace TemporalTransport

instance instIsProbabilityMeasure {Ω : Type*} [MeasurableSpace Ω]
    (tt : TemporalTransport Ω) : IsProbabilityMeasure tt.μ :=
  tt.hprob

/-- Delivery ratio: `P(D)`. -/
def DR {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ENNReal :=
  tt.μ tt.D

/-- Temporal reachability: `P(F)`. -/
def ST {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ENNReal :=
  tt.μ tt.F

/-- Transport efficiency: `P(D | F)`, represented as `P(D) / P(F)`. -/
def eta {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ENNReal :=
  tt.μ tt.D / tt.μ tt.F

/-- Real-valued delivery ratio, useful for ratio comparisons. -/
def DRReal {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ℝ :=
  (tt.DR).toReal

/-- Real-valued temporal reachability, useful for ratio comparisons. -/
def STReal {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ℝ :=
  (tt.ST).toReal

/-- Real-valued efficiency. -/
def etaReal {Ω : Type*} [MeasurableSpace Ω] (tt : TemporalTransport Ω) : ℝ :=
  tt.DRReal / tt.STReal

end TemporalTransport

/--
An augmentation keeps the same event space and preserves all previously
feasible samples.
-/
structure Augmentation {Ω : Type*} [MeasurableSpace Ω]
    (tt tt' : TemporalTransport Ω) where
  h_F_mono : tt.F ⊆ tt'.F

end TINProofs.C4

end
```

TINProofs/C4/Inclusion.lean

```
/-
  C4 Lemma 2.1 consequences: delivery implies feasibility.
-/
import TINProofs.C4.Defs

open MeasureTheory Set

noncomputable section

```

```
namespace TINProofs.C4

variable {Ω : Type*} [MeasurableSpace Ω]

/-- A delivered but infeasible sample cannot occur. -/
theorem delivery_inter_infeasible_eq_empty (tt : TemporalTransport Ω) :
    tt.D ∩ tt.Fᶜ = ∅ := by
  ext x
  constructor
  · intro hx
    exact False.elim (hx.2 (tt.h_incl hx.1))
  · intro hx
    exact False.elim hx

/-- `D ⊆ F` implies `P(D ∩ Fᶜ) = 0`. -/
theorem delivery_zero_when_infeasible (tt : TemporalTransport Ω) :
    tt.μ (tt.D ∩ tt.Fᶜ) = 0 := by
  rw [delivery_inter_infeasible_eq_empty tt]
  exact measure_empty

/-- Delivery probability is bounded by feasibility probability. -/
theorem delivery_measure_le_feasible (tt : TemporalTransport Ω) :
    tt.μ tt.D ≤ tt.μ tt.F :=
  measure_mono tt.h_incl

end TINProofs.C4

end
```

TINProofs/C4/Factorization.lean

```
/-
  C4 Proposition 2.2: delivery ratio factors through temporal reachability
  and conditional efficiency.
-/
import TINProofs.C4.Inclusion

open MeasureTheory Set

noncomputable section

namespace TINProofs.C4

variable {Ω : Type*} [MeasurableSpace Ω]

/-- Exact `ENNReal` factorization: `DR = S_T * eta`. -/
theorem exact_factorization (tt : TemporalTransport Ω) (hF : tt.ST ≠ 0) :
    tt.DR = tt.ST * tt.eta := by
  have hF_ne_zero : tt.μ tt.F ≠ 0 := by
    simpa [TemporalTransport.ST] using hF
  unfold TemporalTransport.DR TemporalTransport.ST TemporalTransport.eta
  rw [ENNReal.mul_div_cancel hF_ne_zero (measure_ne_top tt.μ tt.F)]

/-- Real-valued version of the same factorization. -/
theorem exact_factorization_real (tt : TemporalTransport Ω) (hF : tt.STReal ≠ 0) :
    tt.DRReal = tt.STReal * tt.etaReal := by
  rw [TemporalTransport.etaReal]
  exact (mul_div_cancel₀ tt.DRReal hF).symm

end TINProofs.C4

end
```

TINProofs/C4/Monotonicity.lean

```
/-
  C4 Lemma 2.6: temporal reachability is monotone under augmentation.
-/
import TINProofs.C4.Factorization

open MeasureTheory Set

noncomputable section

namespace TINProofs.C4

variable {Ω : Type*} [MeasurableSpace Ω]

/-- `S_T` is monotone under augmentation. -/
theorem st_monotone (tt tt' : TemporalTransport Ω)
    (h_same_measure : tt.μ = tt'.μ)
    (aug : Augmentation tt tt') :
    tt.ST ≤ tt'.ST := by
  unfold TemporalTransport.ST
  rw [h_same_measure]
  exact measure_mono aug.h_F_mono

/-- Real-valued `S_T` is monotone under augmentation. -/
```

```
theorem stReal_monotone (tt tt' : TemporalTransport Ω)
    (h_same_measure : tt.μ = tt'.μ)
    (aug : Augmentation tt tt') :
    tt.STReal ≤ tt'.STReal := by
  have hmono : tt.ST ≤ tt'.ST := st_monotone tt tt' h_same_measure aug
  have hfinite : tt'.ST ≠ ⊤ := by
    simp [TemporalTransport.ST, measure_ne_top tt'.μ tt'.F]
  simp [TemporalTransport.STReal, ENNReal.toReal_mono hfinite hmono]

end TINProofs.C4

end
```

TINProofs/C4/BraessLocalization.lean

```
/-
  C4 Remark 2.8: any Braess-style delivery-ratio decrease under augmentation
  is localized in the efficiency factor.
-/
import TINProofs.C4.Monotonicity

open MeasureTheory Set

noncomputable section

namespace TINProofs.C4

variable {Ω : Type*} [MeasurableSpace Ω]

/-- If delivery ratio decreases under augmentation, efficiency decreases. -/
theorem braess_in_eta (tt tt' : TemporalTransport Ω)
    (h_same_measure : tt.μ = tt'.μ)
    (aug : Augmentation tt tt')
    (hF : 0 < (tt.μ tt.F).toReal)
    (_hF' : 0 < (tt'.μ tt'.F).toReal)
    (h_dr_decrease : (tt'.μ tt'.D).toReal < (tt.μ tt.D).toReal) :
    (tt'.μ tt'.D).toReal / (tt'.μ tt'.F).toReal <
      (tt.μ tt.D).toReal / (tt.μ tt.F).toReal := by
  have hST_mono : (tt.μ tt.F).toReal ≤ (tt'.μ tt'.F).toReal := by
    simpa [TemporalTransport.STReal, TemporalTransport.ST] using
      stReal_monotone tt tt' h_same_measure aug
  have hDR_nonneg : 0 ≤ (tt.μ tt.D).toReal := ENNReal.toReal_nonneg
  exact div_lt_div₀ h_dr_decrease hST_mono hDR_nonneg hF

/-- Same localization statement in the named real-valued factors. -/
theorem braess_in_etaReal (tt tt' : TemporalTransport Ω)
    (h_same_measure : tt.μ = tt'.μ)
    (aug : Augmentation tt tt')
    (hF : 0 < tt.STReal)
    (_hF' : 0 < tt'.STReal)
    (h_dr_decrease : tt'.DRReal < tt.DRReal) :
    tt'.etaReal < tt.etaReal := by
  unfold TemporalTransport.etaReal
  exact div_lt_div₀ h_dr_decrease
    (stReal_monotone tt tt' h_same_measure aug)
    ENNReal.toReal_nonneg hF

end TINProofs.C4

end
```

Appendix C5: Three-Factor Sparse Law

TINProofs/C5/Defs.lean

```
/-
  C5: Three-factor sparse law -- scalar definitions.
-/
import TINProofs.C4.Factorization

open MeasureTheory Real

noncomputable section

namespace TINProofs.C5

/-- Scalar parameters for the three-factor sparse law. -/
structure SparseLawSetup where
  S_T : ℝ
  eta : ℝ
  E_H : ℝ
  lyap : ℝ
  hST_pos : 0 < S_T
  hST_le : S_T ≤ 1
  heta_nonneg : 0 ≤ eta
  heta_le : eta ≤ 1
  hEH_pos : 0 < E_H
  hlyap_neg : lyap < 0
  DR : ℝ
  hDR : DR = S_T * eta

/-- Chain attenuation factor: `exp(E[H] * lambda)`. -/
def SparseLawSetup.etaLyap (s : SparseLawSetup) : ℝ :=
  Real.exp (s.E_H * s.lyap)

/-- Routing distortion: `Phi = eta / eta_lyap`. -/
def SparseLawSetup.Phi (s : SparseLawSetup) : ℝ :=
  s.eta / s.etaLyap

end TINProofs.C5

end
```

TINProofs/C5/ThreeFactor.lean

```
/-
  C5 Proposition 2.7: exact three-factor sparse law.
-/
import TINProofs.C5.Defs

open MeasureTheory Real

noncomputable section

namespace TINProofs.C5

/-- The three-factor sparse law: `DR = S_T * exp(E[H] * lambda) * Phi`. -/
theorem three_factor (s : SparseLawSetup) :
    s.DR = s.S_T * s.etaLyap * s.Phi := by
  have hpos : 0 < s.etaLyap := by
    unfold SparseLawSetup.etaLyap
    exact Real.exp_pos _
  have heta : s.eta = s.etaLyap * s.Phi := by
    unfold SparseLawSetup.Phi
    exact (mul_div_cancel₀ s.eta (ne_of_gt hpos)).symm
  rw [s.hDR, heta, mul_assoc]

end TINProofs.C5

end
```

TINProofs/C5/ChainProperties.lean

```
/-
  C5: elementary properties of the sparse-law factors.
-/
import TINProofs.C5.Defs

open MeasureTheory Real

noncomputable section

namespace TINProofs.C5

/-- Chain attenuation is positive. -/
theorem etaLyap_pos (s : SparseLawSetup) : 0 < s.etaLyap := by
```

```
  unfold SparseLawSetup.etaLyap
  exact Real.exp_pos _

/-- Chain attenuation is less than one when `E_H > 0` and `lyap < 0`. -/
theorem etaLyap_lt_one (s : SparseLawSetup) : s.etaLyap < 1 := by
  unfold SparseLawSetup.etaLyap
  rw [← Real.exp_zero]
  exact Real.exp_lt_exp.mpr (by nlinarith [s.hEH_pos, s.hlyap_neg])

/-- `Phi` is positive when `eta` is positive. -/
theorem phi_pos (s : SparseLawSetup) (heta : 0 < s.eta) : 0 < s.Phi := by
  unfold SparseLawSetup.Phi
  exact div_pos heta (etaLyap_pos s)

/-- The mean of `log p_h` is negative when every hop probability lies in `(0, 1)`. -/
theorem lyapunov_exponent_neg {n : ℕ} (ps : Fin n → ℝ)
    (hps_pos : ∀ i, 0 < ps i) (hps_lt : ∀ i, ps i < 1)
    (hn : 0 < n) :
    (∑ i, Real.log (ps i)) / (n : ℝ) < 0 := by
  have hsum_neg : (∑ i : Fin n, Real.log (ps i)) < 0 := by
    have hnonempty : (Finset.univ : Finset (Fin n)).Nonempty :=
      ⟨⟨0, hn⟩, by simp⟩
    exact Finset.sum_neg
      (fun i _ => Real.log_neg (hps_pos i) (hps_lt i))
      hnonempty
  have hn_pos : 0 < (n : ℝ) := by
    exact_mod_cast hn
  exact div_neg_of_neg_of_pos hsum_neg hn_pos

end TINProofs.C5

end
```

TINProofs/C5/Classification.lean

```
/-
  C5: sparse-law morphology classification vocabulary.
-/
import TINProofs.C5.ChainProperties

open MeasureTheory Real

noncomputable section

namespace TINProofs.C5

/-- Finite-difference morphology slope for the routing residual. -/
def morphologySlope (phi1 phi2 : ℝ) (eh1 eh2 : ℝ)
    (_hphi1 : 0 < phi1) (_hphi2 : 0 < phi2) (_heh : eh1 ≠ eh2) : ℝ :=
  (Real.log phi2 - Real.log phi1) / (eh2 - eh1)

/-- Trap class: adding hops decreases the residual. -/
def isTrap (gamma : ℝ) : Prop :=
  gamma < 0

/-- Cluster class: adding hops increases the residual. -/
def isCluster (gamma : ℝ) : Prop :=
  0 < gamma

end TINProofs.C5

end
```